

Software Engineering (STE)

Course Code : 315323

Lecturer – Asad Shamna

Semester – 5th

I. RATIONALE

Software Engineering is the foundation for professional processes to be followed for designing, developing, testing and maintaining software involving principles, different techniques, and practices for software development. This course enable students to develop requisite abilities to follow systemic and disciplined approach to software development that aims to create high quality, reliable and maintainable software.

II. INDUSTRY / EMPLOYER EXPECTED OUTCOME

Apply software engineering principles to develop software product.

III. COURSE LEVEL LEARNING OUTCOMES (COS)

Students will be able to achieve & demonstrate the following COs on completion of course based learning

CO1 – Select suitable software development process model

CO2 – Prepare software requirement specification.

CO3 – Construct different Software design models

CO4 – Apply different planning and cost estimation techniques for a software product

CO5 – Apply project management techniques in software development.

CO6 – Use quality assurance principles in software development

UNIT - III SOFTWARE MODELLING AND DESIGN [16M]

3.1 Translating Requirement model into design model: Data Modelling.

3.2 Analysis Modelling: Elements of Analysis model.

3.3 Design modelling: Fundamental Design Concepts (Abstraction, Information hiding, Structure, Modularity, Concurrency, Verification, Aesthetics).

3.4 Design notations: Data Flow Diagram (DFD), Structured Flowcharts, Decision Tables.

3.5 UML Modelling :Use-Case ,Class Diagrams, Sequence Diagrams.

3.6 Testing - Meaning and purpose, testing methods - Black-box and White-box, Static and Dynamic testing, Level of testing, V-model.

3.1 Translating Requirement model into design model: Data Modelling.

- Translating a requirement model into a design model, specifically for data modeling, involves transforming functional and non-functional requirements into a detailed blueprint for the system's data structures, which then guides implementation.
- Key steps include identifying and refining design elements like entities, attributes, and relationships, creating data models (e.g., logical and physical data models) to define these elements and their constraints, allocating responsibilities for data management to components, and ensuring traceability back to the original requirements.

Steps for Data Modeling Translation

1. Understand and Refine Requirements:

- Thoroughly understand the functional and non-functional requirements from the requirement model.
- Refine and expand these high-level requirements to create more detailed design requirements, focusing on the "what" the system needs to do with data.

3.1 Translating Requirement model into design model: Data Modelling.

Steps for Data Modeling Translation

2. Identify and Define Design Elements:

- Identify the core data elements (entities) that represent real-world objects or concepts described in the requirements.
- Determine the attributes (columns) for each entity and the relationships (e.g., one-to-many, many-to-many) between them.
- Consider identifying primary and foreign keys to enforce data integrity.

3. Create Data Models:

- **Logical Data Model:** Develop a detailed, technology-independent model that outlines entities, their attributes, and relationships, including normalization to ensure data structure is sound.
- **Physical Data Model:** Transform the logical model into a physical design by adding details like specific data types for columns, nullability, indexes, and defining how tables and columns will actually be named in the database.

4. Allocate Responsibilities:

- Determine which modules or components of the system will be responsible for managing specific data elements and interactions.

3.1 Translating Requirement model into design model: Data Modelling.

Steps for Data Modeling Translation

5. Ensure Traceability:

- Throughout the process, maintain traceability by linking each design decision and element back to the corresponding requirement in the requirement model.

6. Validation and Refinement:

- Validate the design to ensure it adequately addresses all requirements.
- Iteratively refine the data model based on feedback and changes, ensuring the design remains adaptable to future needs.

3.2 Analysis Modelling:

- Analysis Model is a technical representation of the system. It acts as a link between the system description and the design model. In Analysis Modelling, information, behavior, and functions of the system are defined and translated into the architecture, component, and interface level design in the design modeling.
- An analysis model in software engineering is a technical representation of the system's requirements, translating customer needs into a structured format of diagrams and text to depict the information, functions, and behavior of the software to be built.
- This model acts as a crucial link between the initial high-level system description and the detailed software design. It helps stakeholders and engineers understand the system's requirements, uncover ambiguities, and validate that the final software will meet the desired specifications.

3.2 Analysis Modelling: Purpose and Goals

- **Clarify Requirements:** Converts vague user requirements into a structured, understandable format.
- **Bridge the Gap:** Connects the system-level functionality with detailed software design.
- **Ensure Understanding:** Provides a clear, graphical view of the system for both technical and non-technical stakeholders.
- **Foundation for Design:** Serves as the basis for creating the architecture, components, and interfaces of the software.
- **Validation:** Establishes a basis for validating the software's requirements once it is built.

3.2 Analysis Modelling: Key Elements and Domains

Analysis models typically represent three main aspects of the software:

- **Information Domain:** Describes the data that flows into, through, and out of the system, including data storage.
- **Functional Domain:** Defines the processes and functions the software will perform to meet user needs and provide internal support.
- **Behavioral Domain:** Outlines the system's response to external stimuli, such as user inputs or interactions with other systems.

3.2 Analysis Modelling: Types of Analysis Modeling

Different modeling techniques are used to build analysis models:

- **Scenario-Based Modeling:** Uses use cases to describe the system from the user's perspective.
- **Class-Based Modeling:** Involves identifying and categorizing objects that the system will manipulate.
- **Flow-Oriented Modeling:** Focuses on the flow of data and control within the system, often using Data Flow Diagrams (DFDs).
- **Behavioral Modeling:** Depicts the dynamic behavior of the system, sometimes using State Transition Diagrams.

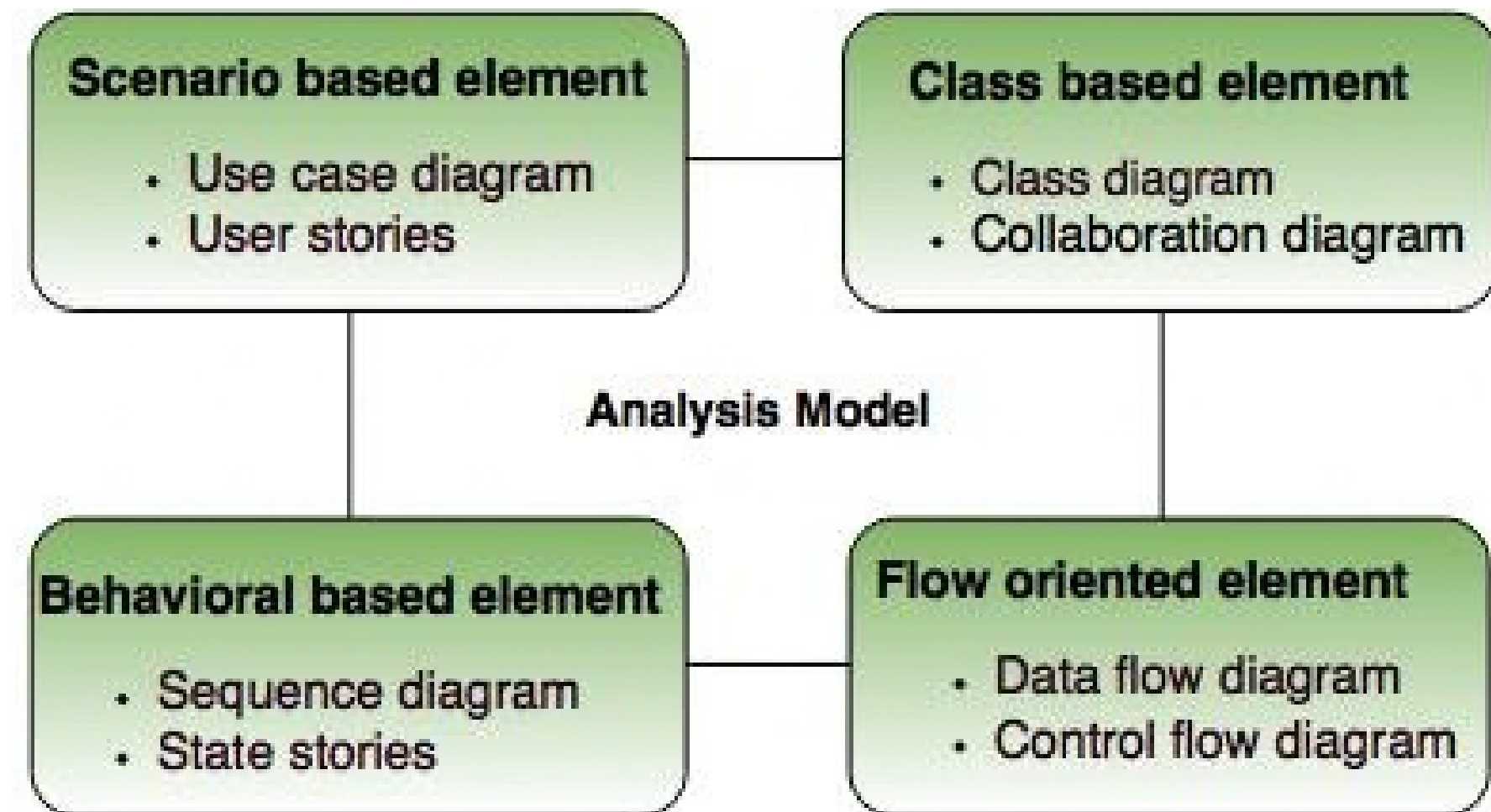
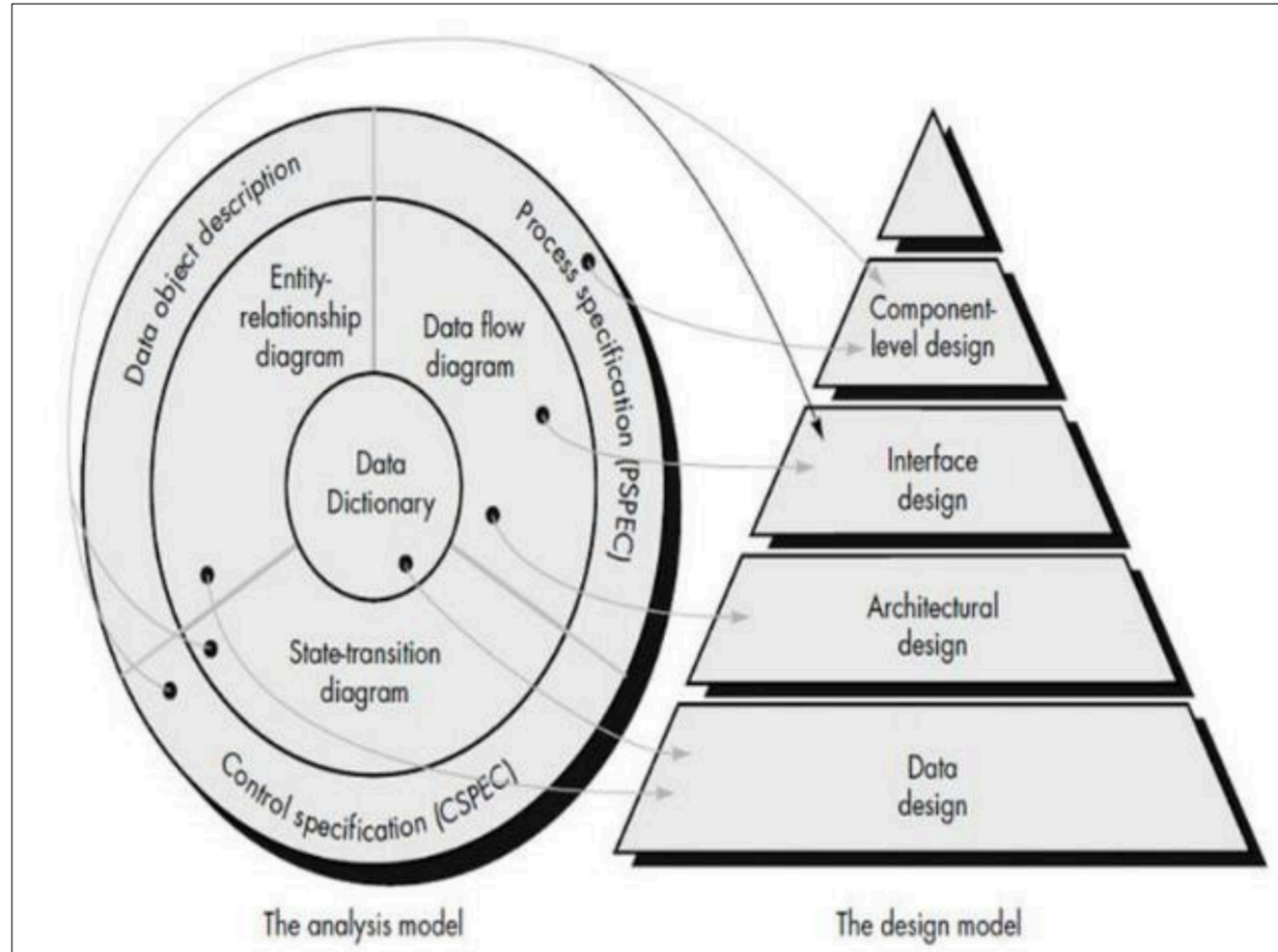
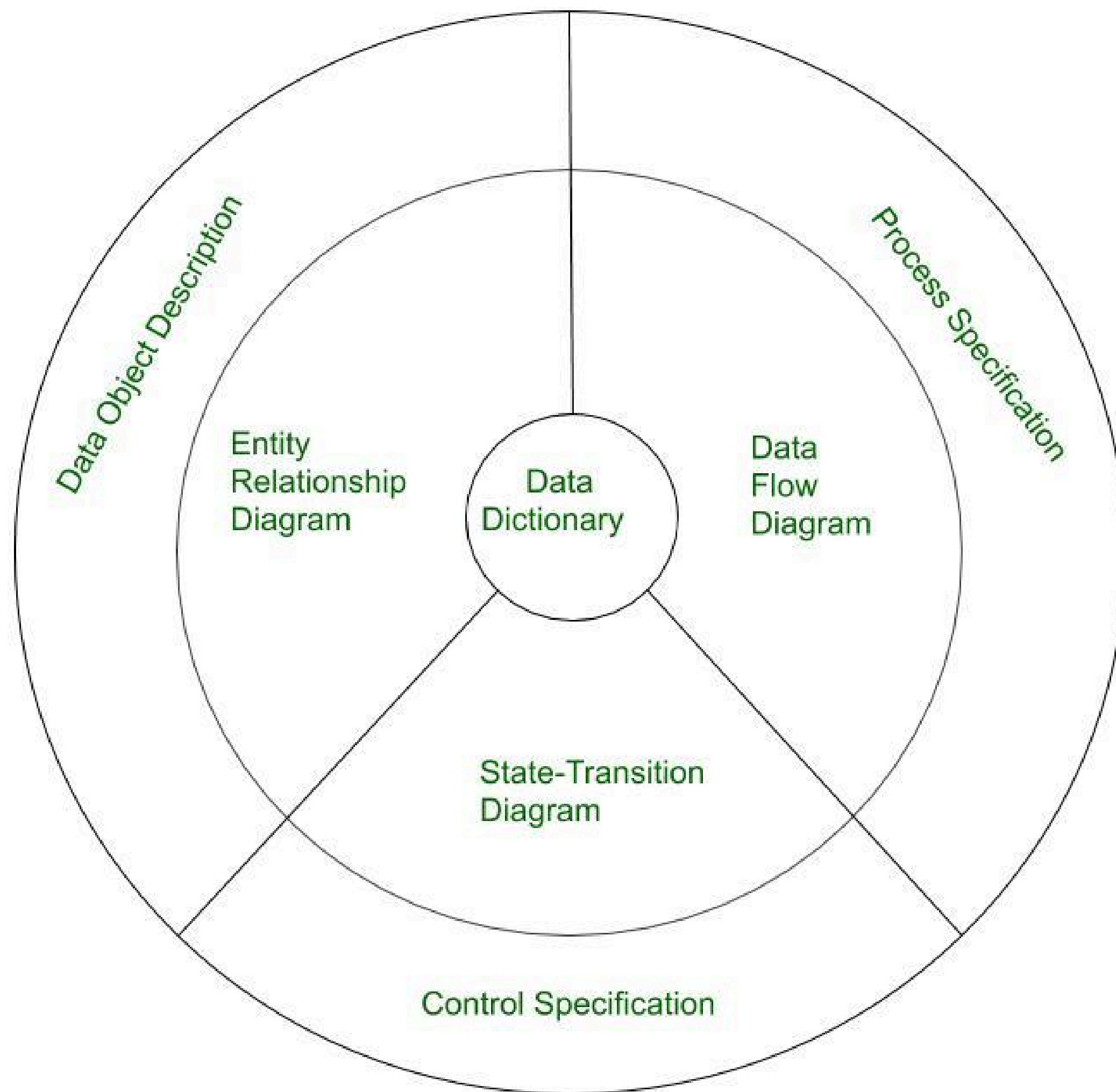


Fig. - Elements of analysis model

3.1 Translating Requirement model into design model: Data Modelling.





3.2.1 Elements of Analysis model.

1. Data Dictionary:

It is a repository that consists of a description of all data objects used or produced by the software. It stores the collection of data present in the software. It is a very crucial element of the analysis model. It acts as a centralized repository and also helps in modeling data objects defined during software requirements.

2. Entity Relationship Diagram (ERD):

It depicts the relationship between data objects and is used in conducting data modeling activities. The attributes of each object in the Entity-Relationship Diagram can be described using Data object description. It provides the basis for activity related to data design.

3. Data Flow Diagram (DFD):

It depicts the functions that transform data flow, and it also shows how data is transformed when moving from input to output. It provides the additional information that is used during the analysis of the information domain and serves as a basis for the modeling of function. It also enables the engineer to develop models of functional and information domains at the same time.

3.2.1 Elements of Analysis model.

4. State Transition Diagram:

It shows various modes of behavior (states) of the system and also shows the transitions from one state to another state in the system. It also provides the details of how the system behaves due to the consequences of external events. It represents the behavior of a system by presenting its states and the events that cause the system to change state. It also describes what actions are taken due to the occurrence of a particular event.

5. Process Specification:

It stores the description of each function present in the data flow diagram. It describes the input to a function, the algorithm that is applied for the transformation of input, and the output that is produced. It also shows regulations and barriers imposed on the performance characteristics that apply to the process and layout constraints that could influence how the process will be implemented.

3.2.1 Elements of Analysis model.

6. Control Specification:

It stores additional information about the control aspects of the software. It is used to indicate how the software behaves when an event occurs and which processes are invoked due to the occurrence of the event. It also provides the details of the processes which are executed to manage events.

7. Data Object Description:

It stores and provides complete knowledge about a data object present and used in the software. It also gives us the details of attributes of the data object present in the Entity Relationship Diagram. Hence, it incorporates all the data objects and their attributes.

3.3 Design modelling:

Design modeling in software engineering is the creation of blueprint-like diagrams to represent a software system's architecture, user interface, and component details, serving as a visual guide before coding begins. It involves breaking the system into manageable parts like data, architecture, user interface, and components, using visual languages like UML to depict structure and behavior, and is essential for clarity, communication, and effective development.

Purpose of Design Modeling

Blueprint for Development:

It acts as a detailed plan for software construction, illustrating how the system will be built.

Enhanced Communication:

Provides a common visual understanding of the system for developers, designers, and stakeholders.

System Visualization:

Simplifies complex software designs into understandable diagrams, making them easier to grasp.

Improved Collaboration:

Allows teams to work on different parts of the system simultaneously, ensuring everyone is on the same page.

3.3 Design modelling: Key Aspects of a Design Model

Design modeling covers several crucial aspects of a software system:

Data Design: Focuses on the structure of data and the relationships between data objects, often using entity–relationship diagrams.

Architectural Design: Outlines the major components of the system and how they interact and communicate.

User Interface (UI) Design: Addresses how users will interact with the system, ensuring a good user experience through intuitive controls and displays.

Component-Level Design: Transforms architectural elements into detailed procedural descriptions of individual software components.

3.3.1 Fundamental Design Concepts

Abstraction:

Hides complex, unnecessary details and presents a simpler, more understandable interface to the user or other developers. .

Information hiding:

Aims to conceal the internal workings and data of a module, preventing unauthorized access and modification.

Structure:

Architecture simply means a technique to design a structure of something. Architecture in designing software is a concept that focuses on various elements and the data of the structure. These components interact with each other and use the data of the structure in architecture.

Modularity:

Decomposes a system into smaller, independent, and manageable components or modules.

Concurrency:

The techniques and principles for designing systems that execute multiple tasks or processes simultaneously, or apparently simultaneously, to improve efficiency, resource utilization, and responsiveness.

3.3.1 Fundamental Design Concepts

Verification:

Verification is the process of checking that software achieves its goal without any bugs. It is the process to ensure whether the product that is developed is right or not. It verifies whether the developed product fulfills the requirements that we have. Verification is static testing. Verification means **Are we building the product right?**

Aesthetics:

- Aesthetic design in software engineering focuses on creating software interfaces and experiences that are visually pleasing, engaging, and intuitive for users.
- The study of beauty and taste is known as aesthetics. Aesthetics refers to the visual appeal and aspects of a design, or an interface.
- The study of aesthetics looks at how people view both natural and artificial sources of experience. It examines what goes through our minds when we interact with things or situations, such as when we watch a fashion show, play, listen to music, read poetry, watch a movie, play, or explore different facets of nature.

3.4 Design notations:

- Software design notations are graphical or textual tools used to plan, describe, and communicate the architecture and logic of a software system, ensuring clarity and facilitating communication between developers and stakeholders.
- Common examples include Structure Charts, which show the organization of modules and data flow between them; Data Flow Diagrams (DFDs), which illustrate how data moves through a system; and modern UML diagrams (like component diagrams), which provide a standardized way to model software components and their relationships.
- Other notations include flowcharts for program logic and wireframes for user interfaces.

Key Types of Design Notations

1. Structure Charts:

What they represent: The static structure of a system, showing modules and how they invoke each other.

How they work: Boxes represent modules, and arrows indicate invocation, with labels showing the data passed between them.

Use: Useful for understanding the hierarchy and dependencies of modules in a program.

2. Data Flow Diagrams (DFDs):

What they represent: How data enters, is processed by, and leaves a system.

How they work: Use a hierarchical series of diagrams to break down a system's functions into sub-functions.

Use: Essential for understanding system processes and identifying data dependencies.

3. User Interface (UI) Notations:

What they represent: The user-facing elements of a software application.

How they work: Notations like wireframes provide a simple visual blueprint of the layout and interaction of user interface elements.

Use: To design and communicate the user experience and layout of an application before coding.

4. Unified Modeling Language (UML):

What it represents: A standardized graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software system.

How it works: Uses various diagrams, such as component diagrams, to show software architecture, internal details of components, and their relationships.

Use: A comprehensive tool for modeling complex software systems in object-oriented development.

3.4.1 Data Flow Diagram (DFD):

In software engineering, a Data Flow Diagram (DFD) is a graphical tool that visualizes how data moves through a system, showing its inputs, outputs, processes, and storage points. DFDs help system analysts and stakeholders understand complex systems by illustrating the flow of information between external entities and internal processes. They consist of four main components: external entities (data sources/sinks), processes (data transformations), data stores (where data is held), and data flows (paths data travels).

Key Aspects of a DFD

Visual Representation:

DFDs offer a simple, graphical way to represent a system's information flow, making complex systems easier to understand.

Focus on Data, Not Control:

Unlike flowcharts, DFDs concentrate solely on the movement and transformation of data, excluding control flow, decision rules, and loops.

System Boundaries:

DFDs define the scope of a system by showing what data enters and exits, as well as where it is transformed and stored.

Components of a DFD

DFDs use standardized symbols for their core components:

- **External Entities:** These are sources or sinks of data, such as users, other systems, or organizations that send data to the system or receive data from it.
- **Processes:** These represent actions that transform input data into output data.
- **Data Stores:** These are places where data is stored or held within the system.
- **Data Flows:** Arrows indicate the direction of data movement between entities, processes, and data stores.

Uses of DFDs

System Analysis: DFDs help in analyzing existing systems to identify inefficiencies and potential areas for improvement.

System Design: They are used during the design phase to plan software architecture and define the flow of information.

Communication: DFDs serve as a common language for communication between system analysts, developers, and non-technical stakeholders.

Documentation: They provide clear documentation for software systems, detailing how data is processed and moved.

Basic Structure of DFD



NOTE:
Red arrows signify flow of data.

Characteristics of Data Flow Diagram (DFD)

Below are some characteristics of Data Flow Diagram (DFD):

1. Graphical Representation:

Data Flow Diagram (DFD) use different symbols and notation to represent data flow within system. That simplify the complex system into understandable visual elements. This makes them easier to interpret by both technical and non-technical stakeholders.

2. Problem Analysis:

Data Flow Diagram (DFDs) are very useful in understanding a system and can be effectively used during analysis. Data Flow Diagram (DFDs) are quite general and are not limited to problem analysis for software requirements specification.

3. Abstraction:

DFDs abstract away the implementation details and focus on the data flow and processes within a system. They provide a high-level overview and omit unnecessary technical information.

4. Hierarchy:

Data Flow Diagram (DFD) provides a hierarchy of a system. High- level diagram i.e. 0-level diagram provides an overview of entire system while lower-level diagram like 1-level DFD and beyond provides a detailed data flow of individual process.

Levels in Data Flow Diagram (DFD)

DFDs are categorized into various levels, with each level providing different degrees of detail. The levels are numbered from 0 and onward. The higher the level, the more detailed the diagram becomes. The following are the four levels of DFDs:

0-Level Data Flow Diagram (DFD)

Level 0 DFD is the highest-level diagram, representing the system as a single process with its interactions with external entities. It shows the major processes, data flows, and data stores in the system, without providing any details about the internal workings of these processes. It is also known as the Context Diagram, which abstracts the system's operations and shows how data enters and leaves the system.

1-Level Data Flow Diagram (DFD)

Level 1 DFD provides a more detailed view of the system by breaking down the major processes identified in the level 0 Data Flow Diagram (DFD) into sub-processes. Each sub-process is depicted as a separate process on the level 1 Data Flow Diagram (DFD). The data flows and data stores associated with each sub-process are also shown. Level 1 DFD provides a more detailed view of the system, focusing on key functional aspects. The Context Diagram from Level 0 is expanded into multiple bubbles/processes.

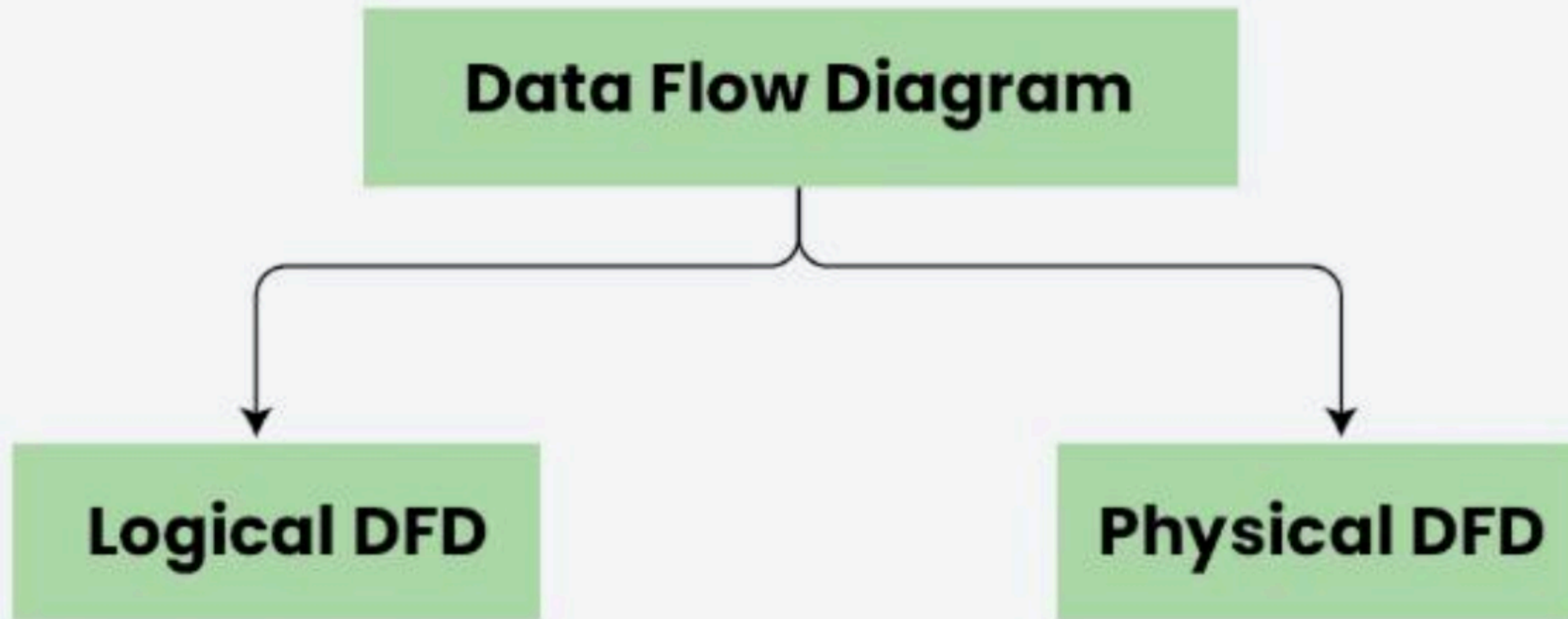
2-Level Data Flow Diagram (DFD)

Level 2 DFD further breaks down the sub-processes from Level 1 DFD into additional sub-processes, providing an even more detailed view. This level is useful when dealing with specific requirements or parts of the system that need a closer examination of their processes and interactions.

3-Level Data Flow Diagram (DFD)

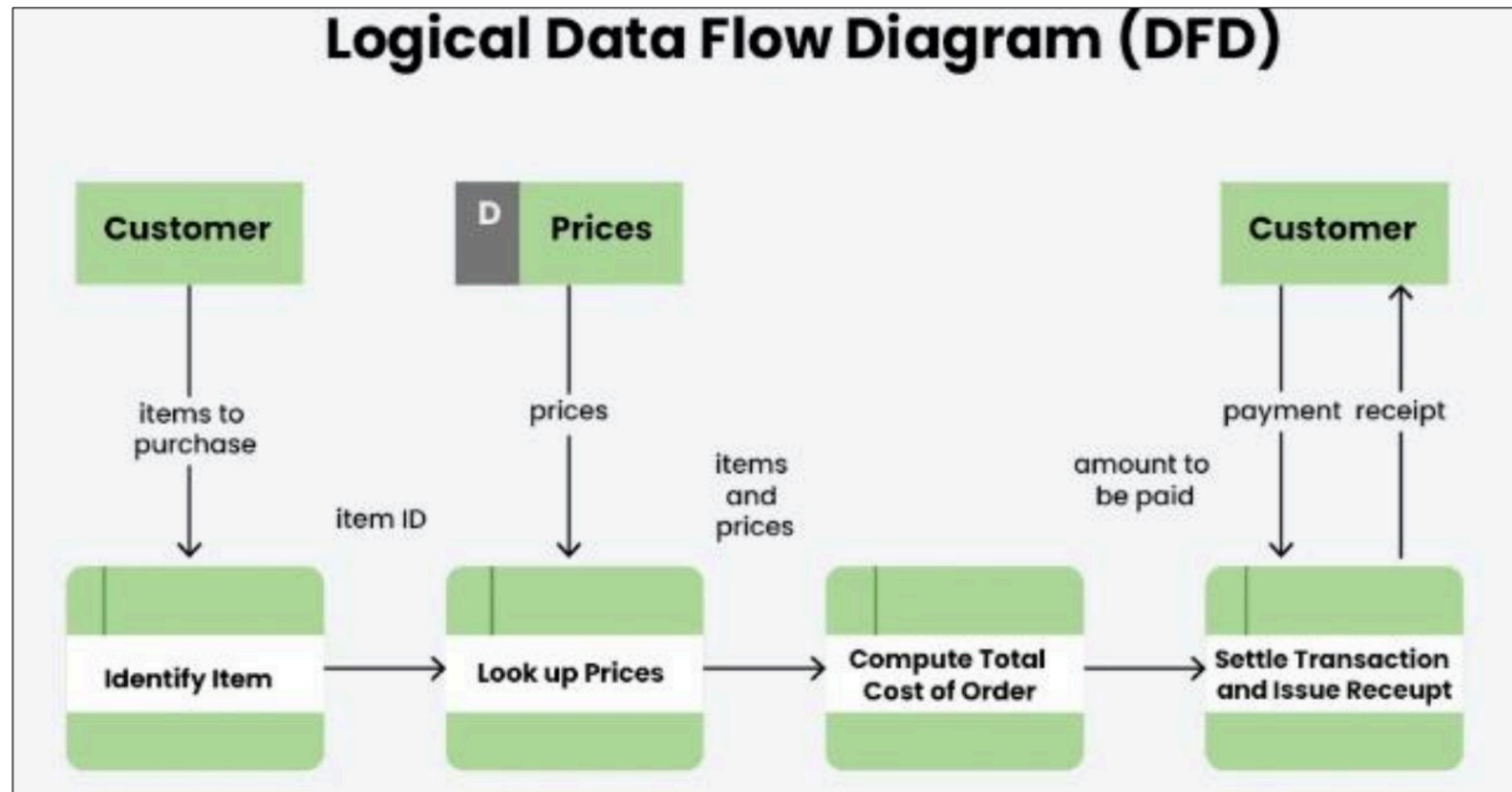
3-Level is the most detailed level of Data Flow Diagram (DFDs), which provides a detailed view of the processes, data flows, and data stores in the system. This level is typically used for complex systems, where a high level of detail is required to understand the system. It includes detailed descriptions of each process, data flow, and data store, and is usually used when there is a need for a comprehensive understanding of the system.

Types of Data Flow Diagram (DFD)



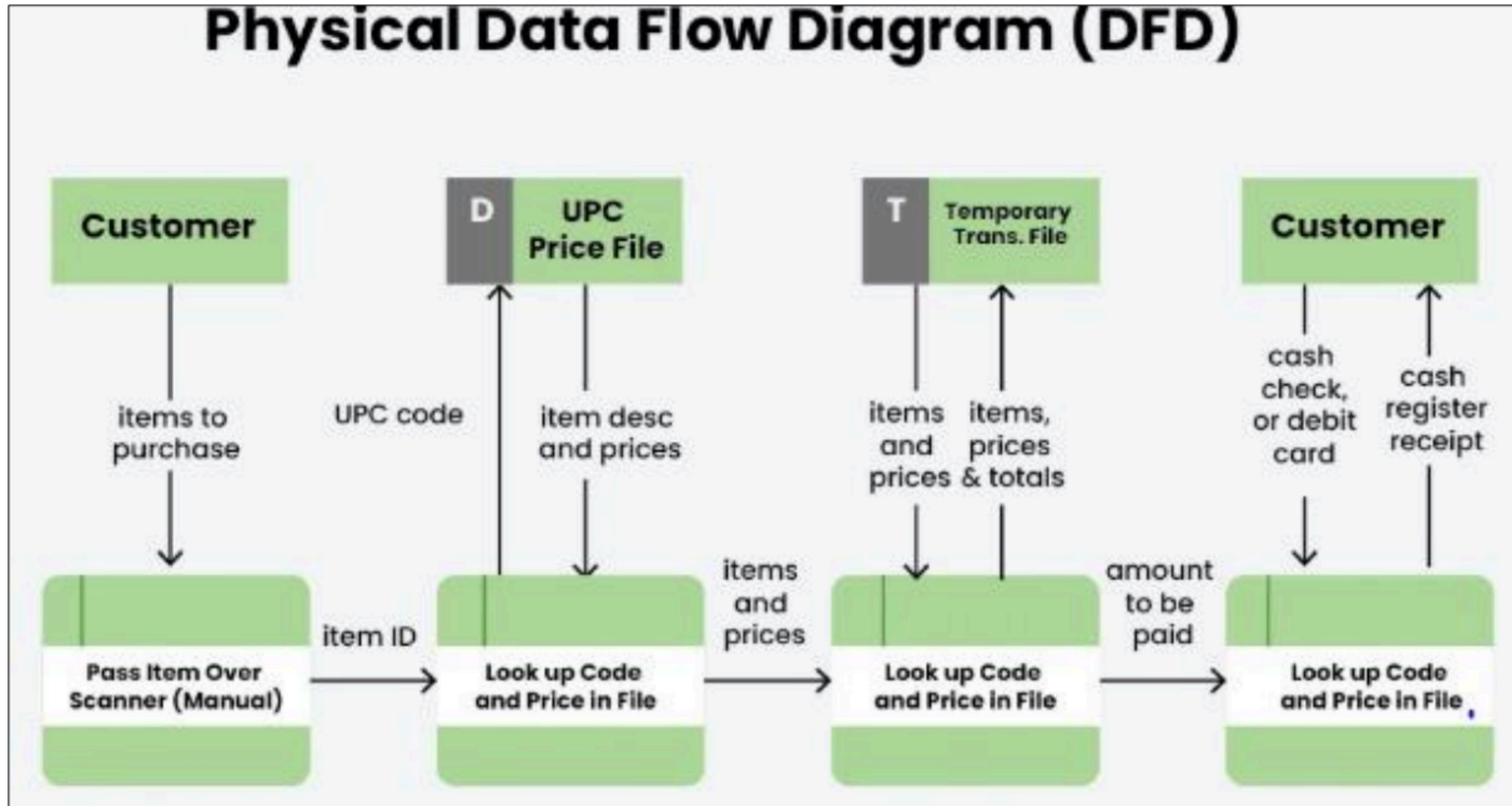
1. Logical Data Flow Diagram (DFD)

The Logical Data Flow Diagram mainly focuses on the system process. It illustrates how data flows in the system. Logical Data Flow Diagram (DFD) mainly focuses on high level processes and data flow without diving deep into technical implementation details. Logical DFDs is used in various organizations for the smooth running of system. Like in a Banking software system, it is used to describe how data is moved from one entity to another.

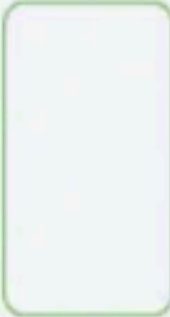
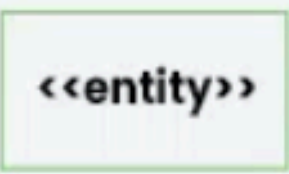
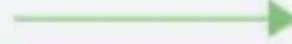


2. Physical Data Flow Diagram

Physical data flow diagram shows how the data flow is actually implemented in the system. In the Physical Data Flow Diagram (DFD), we include additional details such as data storage, data transmission, and specific technology or system components. Physical DFDs are more detailed and provide a closer look at the actual implementation of the system, including the hardware, software, and physical aspects of data processing.



Data Flow Diagram Methods & Symbols

	YOURDON + DE MARCO	ANE & SARSON	SSADM	UNIFIED
External Entry				
Process				
Store				
Flow				

1. Process

- Input to output transformation in a system takes place because of process function. The symbols of a process are rectangular with rounded corners, oval, rectangle or a circle. The process is named a short sentence, in one word or a phrase to express its essence.

2. Data Flow

- Data flow describes the information transferring between different parts of the systems. The arrow symbol is the symbol of data flow. A relatable name should be given to the flow to determine the information which is being moved.
- Data flow also represents material along with information that is being moved. Material shifts are modeled in systems that are not merely informative. A given flow should only transfer a single type of information. The direction of flow is represented by the arrow which can also be bi-directional.

3. Warehouse (Data Store)

- The data is stored in the warehouse for later use. Two horizontal lines represent the symbol of the store. The warehouse is simply not restricted to being a data file rather it can be anything like a folder with documents, an optical disc, a filing cabinet.
- The data warehouse can be viewed independent of its implementation. When the data flow from the warehouse it is considered as data reading and when data flows to the warehouse it is called data entry or data updating.

4. Terminator (External Entity)

The Terminator is an external entity that stands outside of the system and communicates with the system. It can be, for example, organizations like banks, groups of people like customers or different departments of the same organization, which is not a part of the model system and is an external entity. Modeled systems also communicate with terminator.

Rules for Data Flow Diagram (DFD)

Following are the rules of DFD:

1. Data can flow from

Terminator or External Entity → Process

Process → Terminator or External Entity

Process → Data Store

Data Store → Process

Process → Process

2. Data Cannot Flow From

Terminator or External Entity → Terminator or External Entity

Terminator or External Entity → Data Store

Data Store → Terminator or External Entity

Data Store → Data Store

Example:

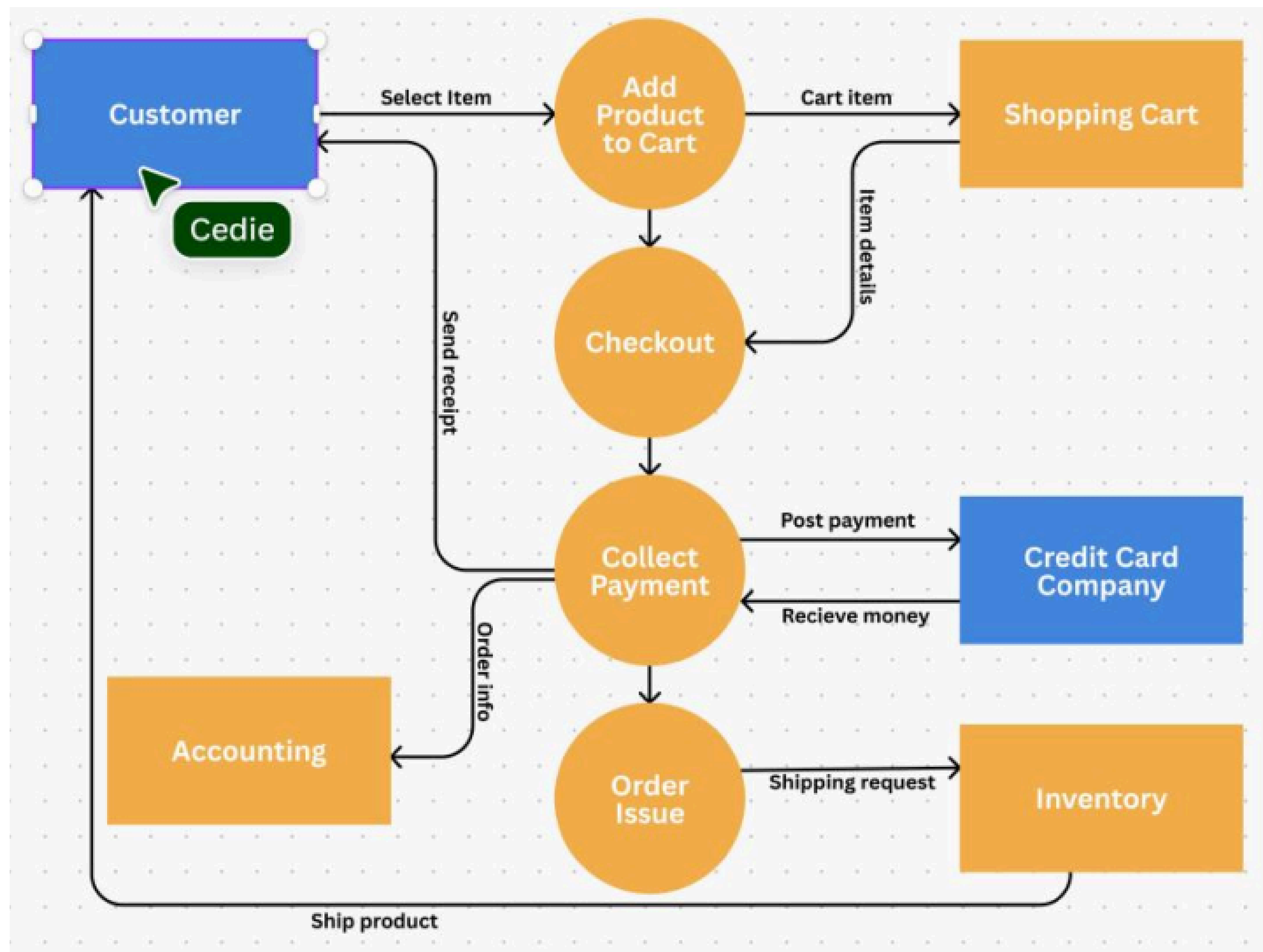
A simple data flow diagram (DFD) example is the "Online Order System". It includes an external entity "Customer," a process "Process Order," a data store "Orders," and another external entity "Shipping Department". Data flows include "Order Details" from the Customer to the Process Order, "Order Confirmation" from the Process Order to the Customer, and "Shipping Request" from Process Order to the Shipping Department.

Components of the DFD:

- **External Entity (Customer):** A source or destination of data that is outside the system.
- **Process (Process Order):** A task that transforms or transforms incoming data into outgoing data.
- **Data Store (Orders):** Where data is held for later use.
- **Data Flow (Arrows):** Shows the movement of data between components.

Step-by-Step Example:

1. Customer: places an order, sending Order Details (data) to the Process Order (system).
2. The Process Order stores the order information by writing to the Orders (data store).
3. It then retrieves the details from the Orders (data store) to prepare the order for fulfillment.
4. The Process Order sends a Shipping Request to the Shipping Department (another external entity).
5. Finally, the Process Order sends an Order Confirmation back to the Customer.



3.4.2 Structured Flowcharts:

Structure Chart represents the hierarchical structure of modules. It breaks down the entire system into the lowest functional modules and describes the functions and sub-functions of each module of a system in greater detail.

What is a Structure Chart?

Structure Chart partitions the system into black boxes (functionality of the system is known to the users, but inner details are unknown).

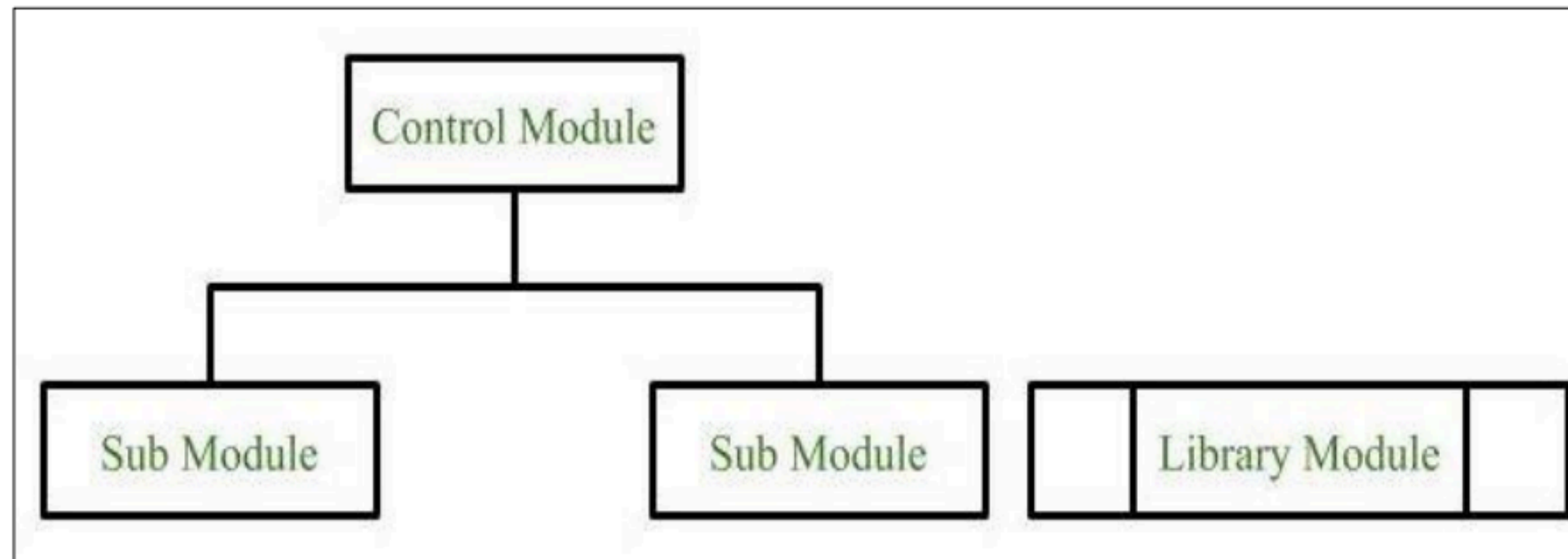
1. Inputs are given to the black boxes and appropriate outputs are generated.
2. Modules at the top level are called modules at low level.
3. Components are read from top to bottom and left to right.
4. When a module calls another, it views the called module as a black box, passing the required parameters and receiving results.

Symbols in Structured Chart

1. Module

It represents the process or task of the system. It is of three types:

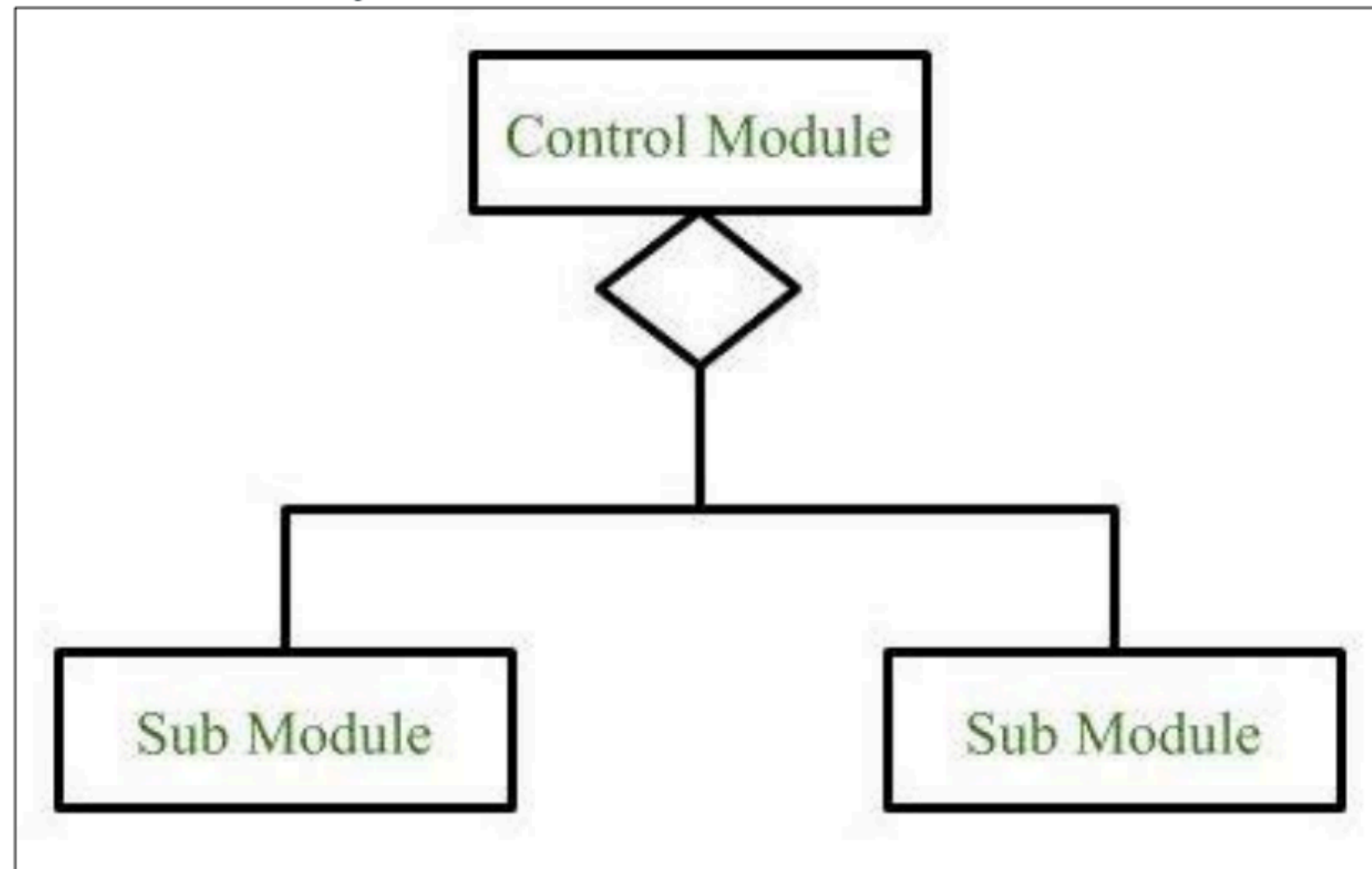
- Control Module: A control module branches to more than one submodule.
- Sub Module: Sub Module is a module which is the part (Child) of another module.
- Library Module: Library Module are reusable and invocable from any module.



Symbols in Structured Chart

2. Conditional Call

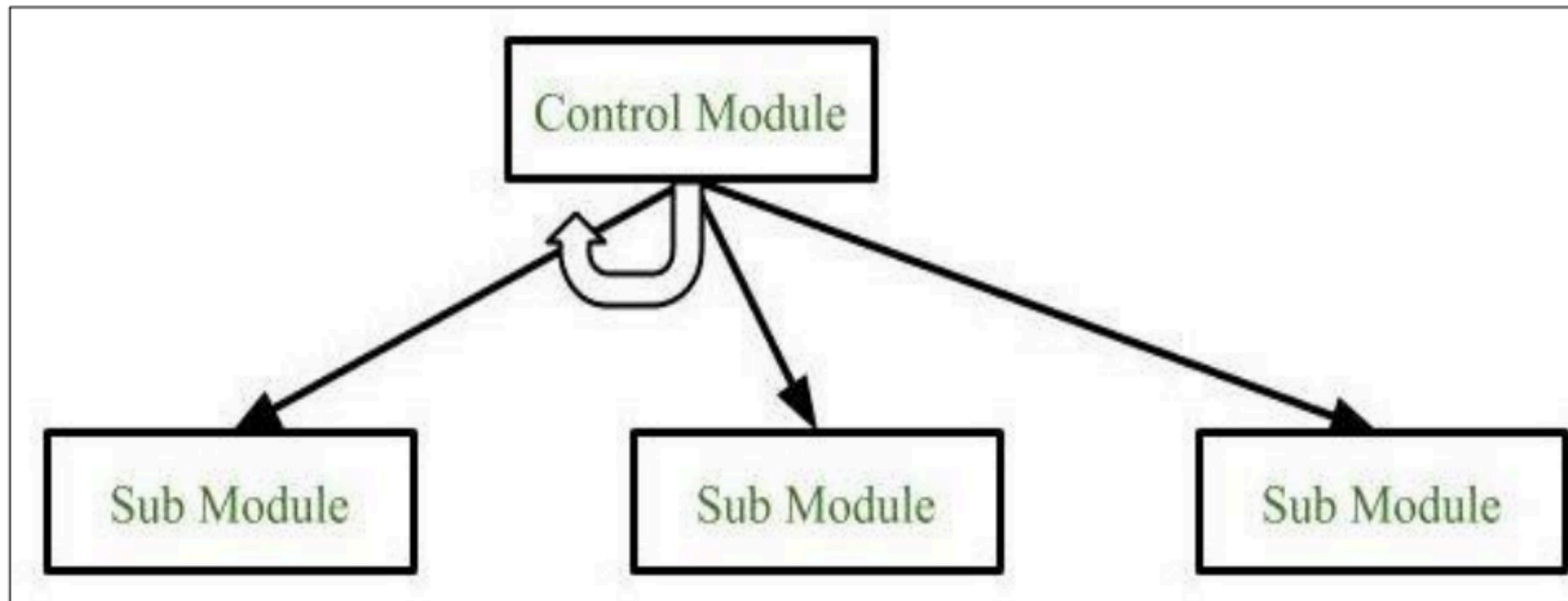
- It represents that control module can select any of the sub module on the basis of some condition.



Symbols in Structured Chart

3. Loop (Repetitive call of module)

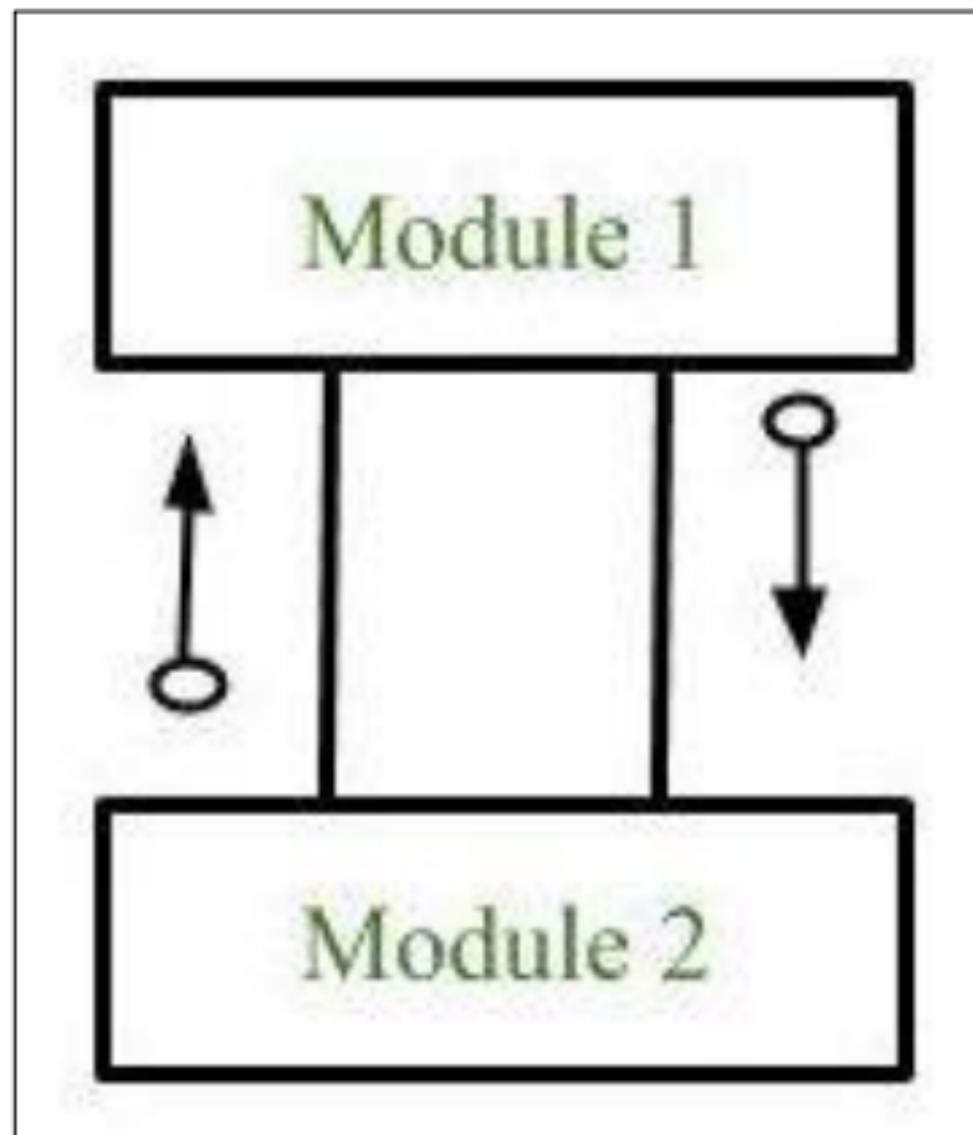
- It represents the repetitive execution of module by the sub module. A curved arrow represents a loop in the module.
- All the submodules cover by the loop repeat execution of module.



Symbols in Structured Chart

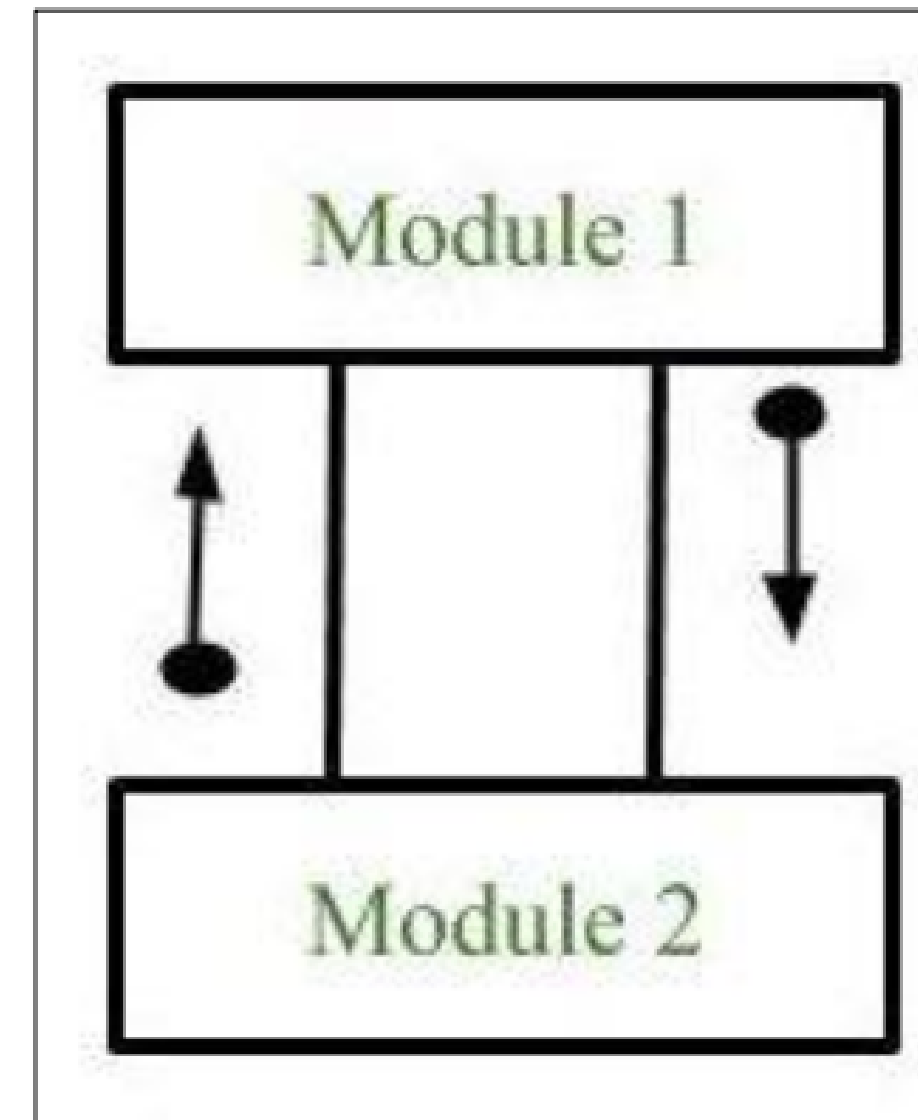
4. Data Flow

- It represents the flow of data between the modules. It is represented by a directed arrow with an empty circle at the end.



5. Control Flow

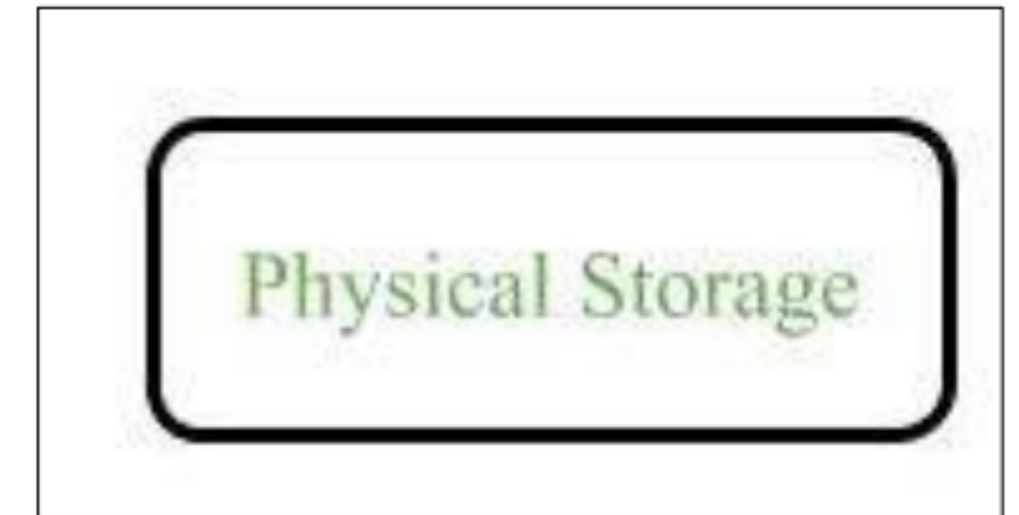
It represents the flow of control between the modules. It is represented by a directed arrow with a filled circle at the end.



Symbols in Structured Chart

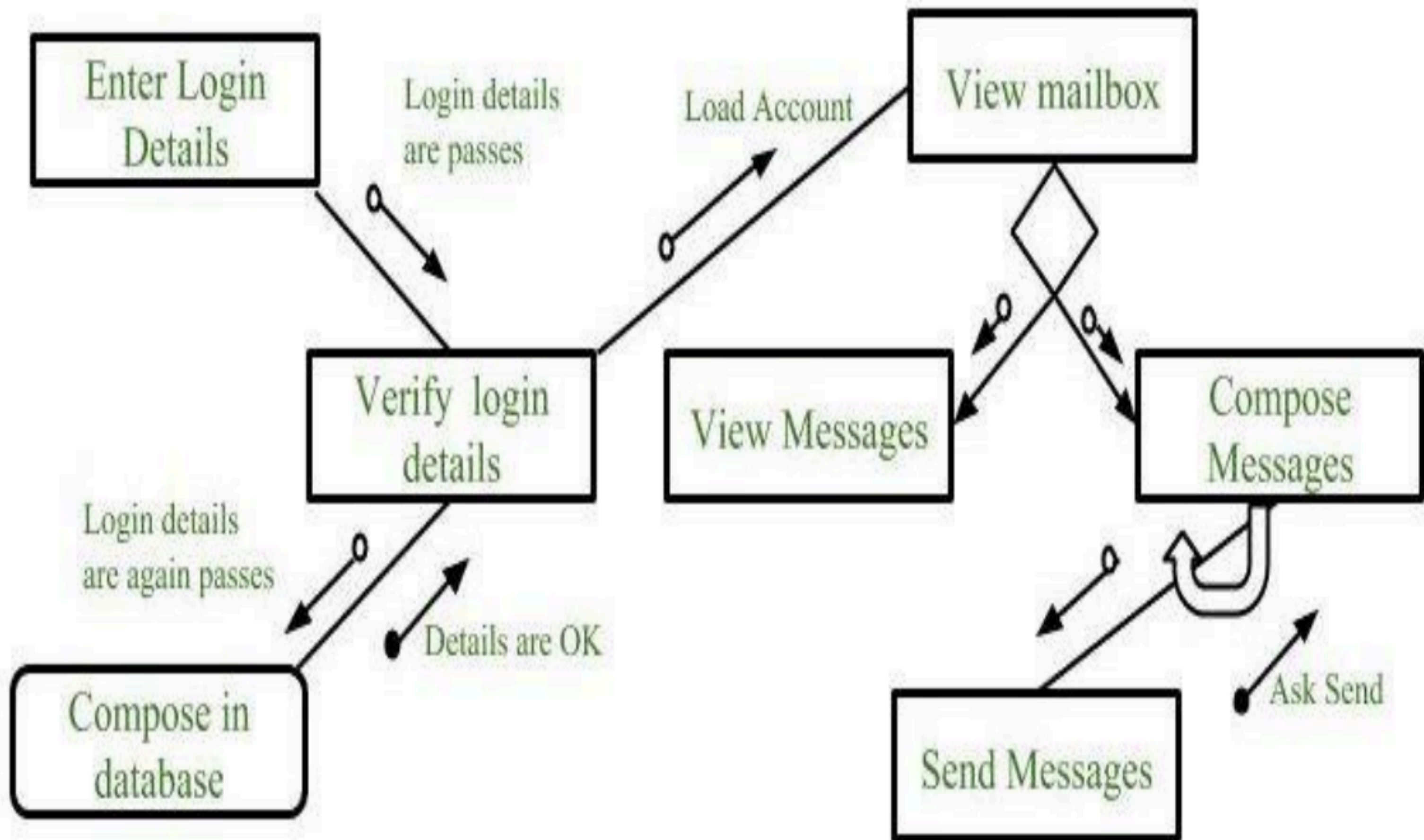
6. Physical Storage

- **It is that where all the information are to be stored.**



Types of Structure Chart

1. Transform Centered Structure: These type of structure chart are designed for the systems that receives an input which is transformed by a sequence of operations being carried out by one module.
2. Transaction Centered Structure: These structure describes a system that processes a number of different types of transaction.



3.4.3 Decision Tables:

- In software engineering, a decision table is a tabular method used to represent complex business rules and logic by mapping various input combinations (conditions) to their corresponding outputs (actions).
- It helps in developing clear requirements, creating comprehensive test cases, identifying potential gaps in logic, and documenting system behavior. Also known as a cause-effect table, it visually organizes conditions and their outcomes, ensuring that all relevant combinations are considered for testing and development.

What is a Decision Table?

A decision table is a structured grid that shows:

- **Conditions:** The various inputs or criteria that the system must evaluate.
- **Actions:** The results or outputs that occur based on the conditions met.
- **Condition Entries:** The combinations of conditions (often represented as True/False) for each rule.
- **Action Entries:** The specific actions taken for each combination of conditions.

Example of transferring money online to an account which is already added and approved.

Here the conditions to transfer money are **ACCOUNT ALREADY APPROVED, OTP (One Time Password) MATCHED, SUFFICIENT MONEY IN THE ACCOUNT.**

And the actions performed are **TRANSFER MONEY, SHOW A MESSAGE AS INSUFFICIENT AMOUNT, BLOCK THE TRANSACTION INCASE OF SUSPICIOUS TRANSACTION.**

Here we decide under what condition the action be performed Now let's see the tabular column below.

ID	CONDITIONS/ACTIONS	TEST CASE 1	TEST CASE 2	TEST CASE 3	TEST CASE 4	TEST CASE 5
Condition 1	Account Already Approved	T	T	T	T	F
Condition 2	OTP (One Time Password) Matched	T	T	F	F	X
Condition 3	Sufficient Money in the Account	T	F	T	F	X
Action 1	Transfer Money	Execute				
Action 2	Show a Message as 'Insufficient Amount'		Execute			
Action 3	Block The Transaction Incase of Suspicious Transaction			Execute	Execute	X

In the first column I took all the conditions and actions related to the requirement.

All the other columns represent Test Cases.

T = True, F = False, X = Not possible

From the case 3 and case 4, we could identify that if condition 2 failed then system will execute Action 3. So we could take either of case 3 or case 4

So finally concluding with the below tabular column.

ID	CONDITIONS/ACTIONS	TEST CASE 1	TEST CASE 2	TEST CASE 3	TEST CASE 4
Condition 1	Account Already Approved	T	T	T	F
Condition 2	OTP (One Time Password) Matched	T	T	F	X
Condition 3	Sufficient Money in the Account	T	F	T	X
Action 1	Transfer Money	Execute			
Action 2	Show a Message as 'Insufficient Amount'		Execute		
Action 3	Block The Transaction Incase of Suspicious Transaction			Execute	X

We write 4 test cases for this requirement.

3.5 Unified Modelling Language (UML):

- UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems.
- We use UML diagrams to show the behavior and structure of a system.
- UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.
- The International Organization for Standardization (ISO) published UML as an approved standard in 2005. UML has been revised over the years and is reviewed periodically.

Types of UML Diagrams commonly used in software engineering:

Structure Diagrams:

Class Diagram: Shows classes, interfaces, and their relationships (inheritance, association, aggregation, composition).

Component Diagram: Illustrates the organization and dependencies among software components.

Deployment Diagram: Models the physical deployment of software artifacts on hardware nodes.

Object Diagram: Shows instances of classes and their relationships at a specific point in time.

Package Diagram: Organizes UML diagrams into meaningful groups and depicts dependencies between packages.

Behavior Diagrams:

- **Use Case Diagram:** Describes the functionality of a system from the user's perspective.
- **Activity Diagram:** Visualizes the flow of activities within a system, similar to a flowchart.
- **Sequence Diagram:** Shows the order of interactions between objects over time.
- **State Machine Diagram:** Models the different states an object can be in and the transitions between them.
- **Communication Diagram:** Focuses on the message flow between objects and their structural relationships.

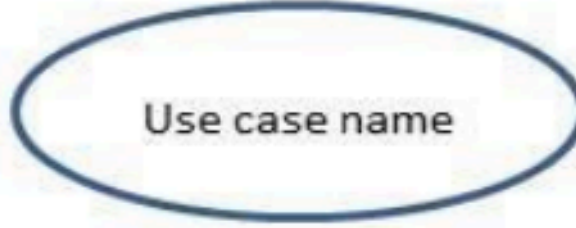
3.5.1 Use-Case:

A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interactions between users (actors) and a system. It captures the functional requirements of a system, showing how different users engage with various use cases, or specific functionalities, within the system.



Actor name

Actor



Use case name

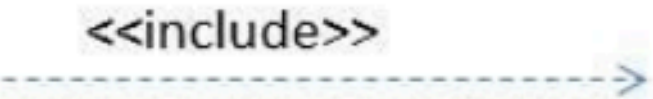
Use case



Generalization symbol used between actors and between use cases

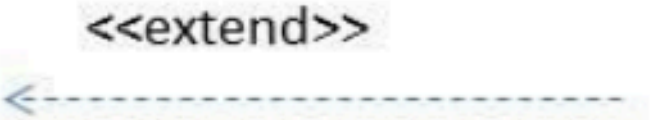


Association between actor and use case



<<include>>

Include relationship between use cases



<<extend>>

Extend relationship between use cases

Symbols in a use case diagram

1. Actor: Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modeling system behavior.

2. Use Case: Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information". Use cases are represented by ovals.

3. Associations are shown as lines between actors and use cases: The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

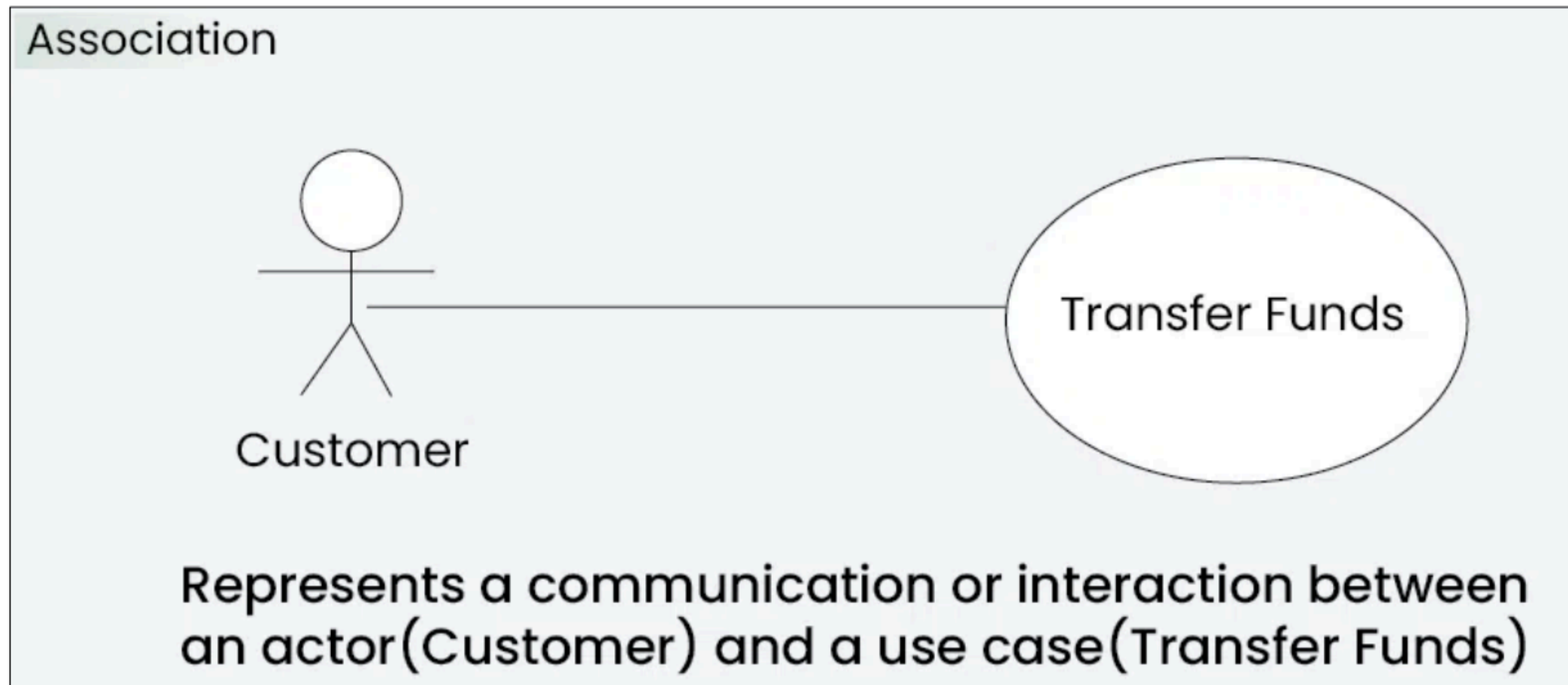
4. System Boundary: The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.

1. Example: Online Banking System

Actor: Customer

Use Case: Transfer Funds

Association: A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.



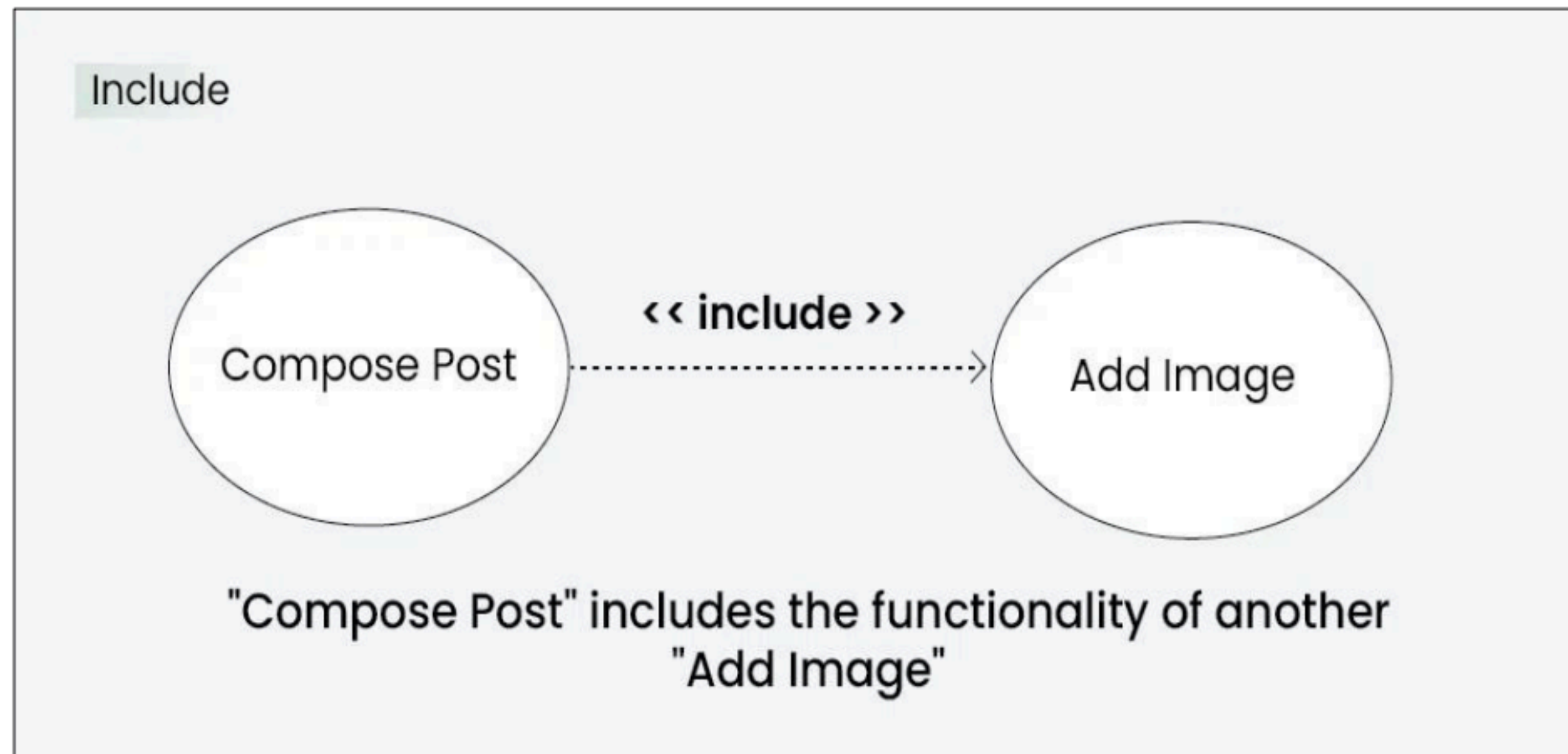
2. Include Relationship

The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case. This relationship promotes modular and reusable design.

Example: Social Media Posting

Use Cases: Compose Post, Add Image

**Include Relationship: The "Compose Post" use case includes the functionality of "Add Image."
Therefore, composing a post includes the action of adding an image.**



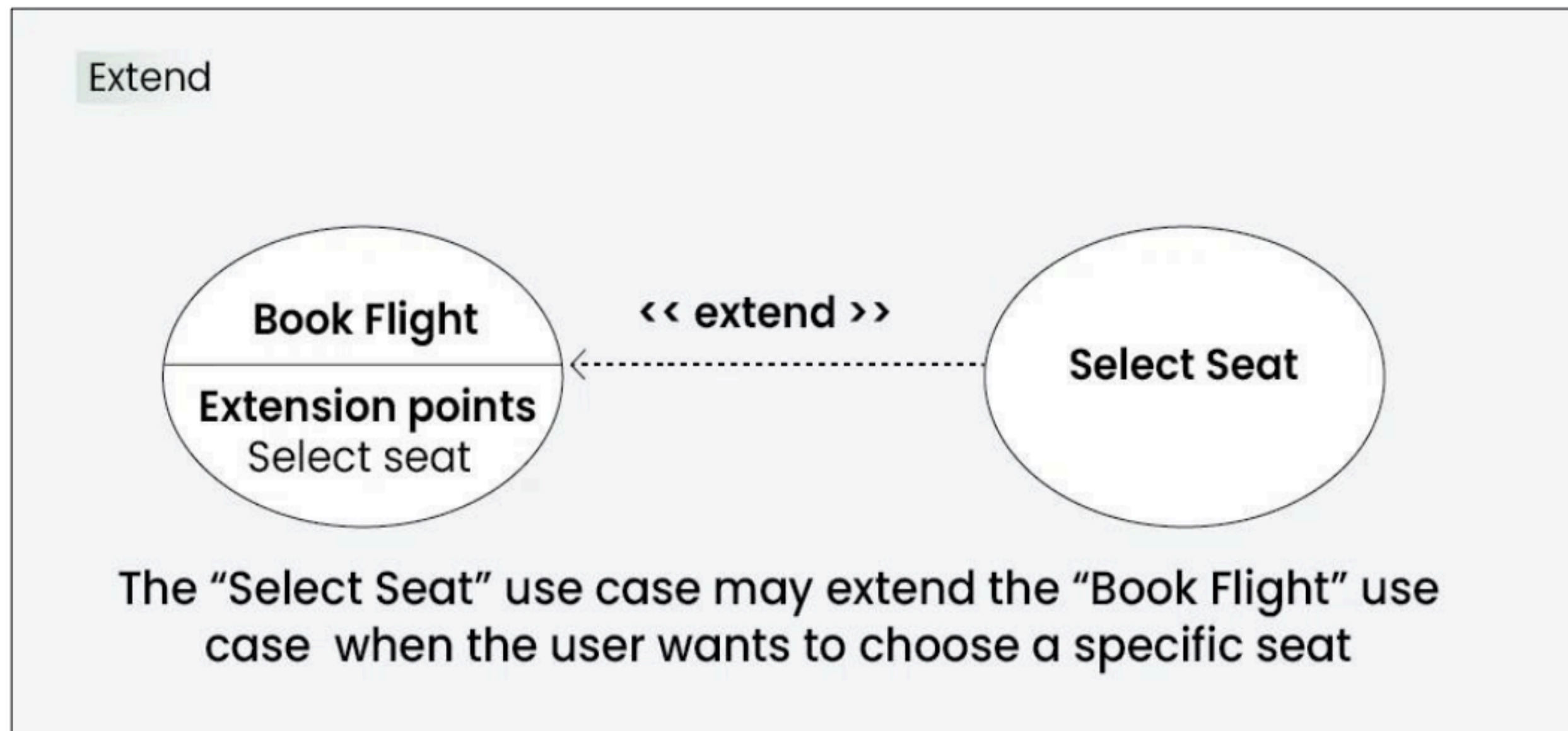
3. Extend Relationship

The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword "extend." This relationship is useful for handling optional or exceptional behavior.

Example: Flight Booking System

Use Cases: Book Flight, Select Seat

Extend Relationship: The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.



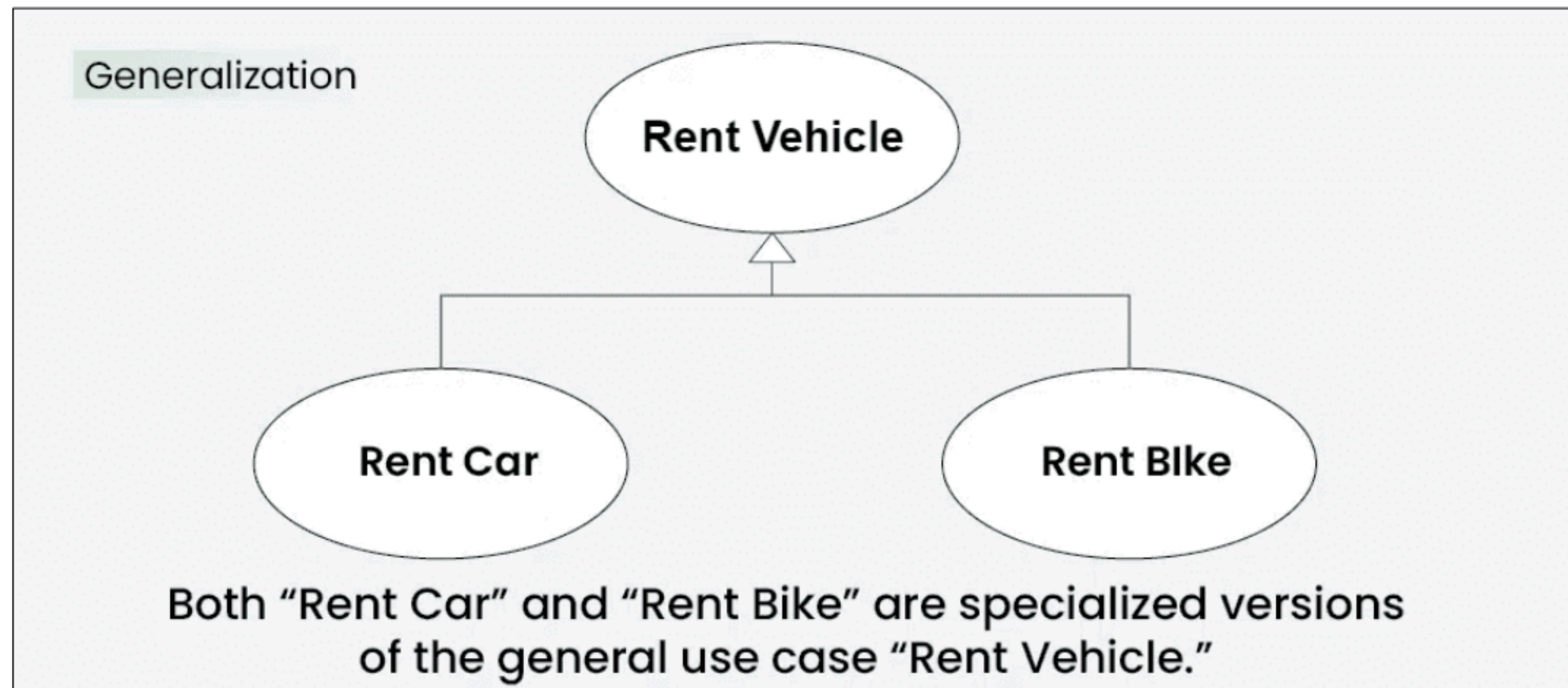
4. Generalization Relationship

The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

Example: Vehicle Rental System

Use Cases: Rent Car, Rent Bike

Generalization Relationship: Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."



How to draw a Use Case diagram in UML?

Below are the main steps to draw use case diagram in UML:

Step 1: Identify Actors: Determine who or what interacts with the system. These are your actors. They can be users, other systems, or external entities.

Step 2: Identify Use Cases: Identify the main functionalities or actions the system must perform. These are your use cases. Each use case should represent a specific piece of functionality.

Step 3: Connect Actors and Use Cases: Draw lines (associations) between actors and the use cases they are involved in. This represents the interactions between actors and the system.

Step 4: Add System Boundary: Draw a box around the actors and use cases to represent the system boundary. This defines the scope of your system.

Step 5: Define Relationships: If certain use cases are related or if one use case is an extension of another, you can indicate these relationships with appropriate notations.

Step 6: Review and Refine: Step back and review your diagram. Ensure that it accurately represents the interactions and relationships in your system. Refine as needed.

Step 7: Validate: Share your use case diagram with stakeholders and gather feedback. Ensure that it aligns with their understanding of the system's functionality.

Use Case Diagram example(Online Shopping System)

Let's understand how to draw a Use Case diagram with the help of an Online Shopping System:

- **Actors:**

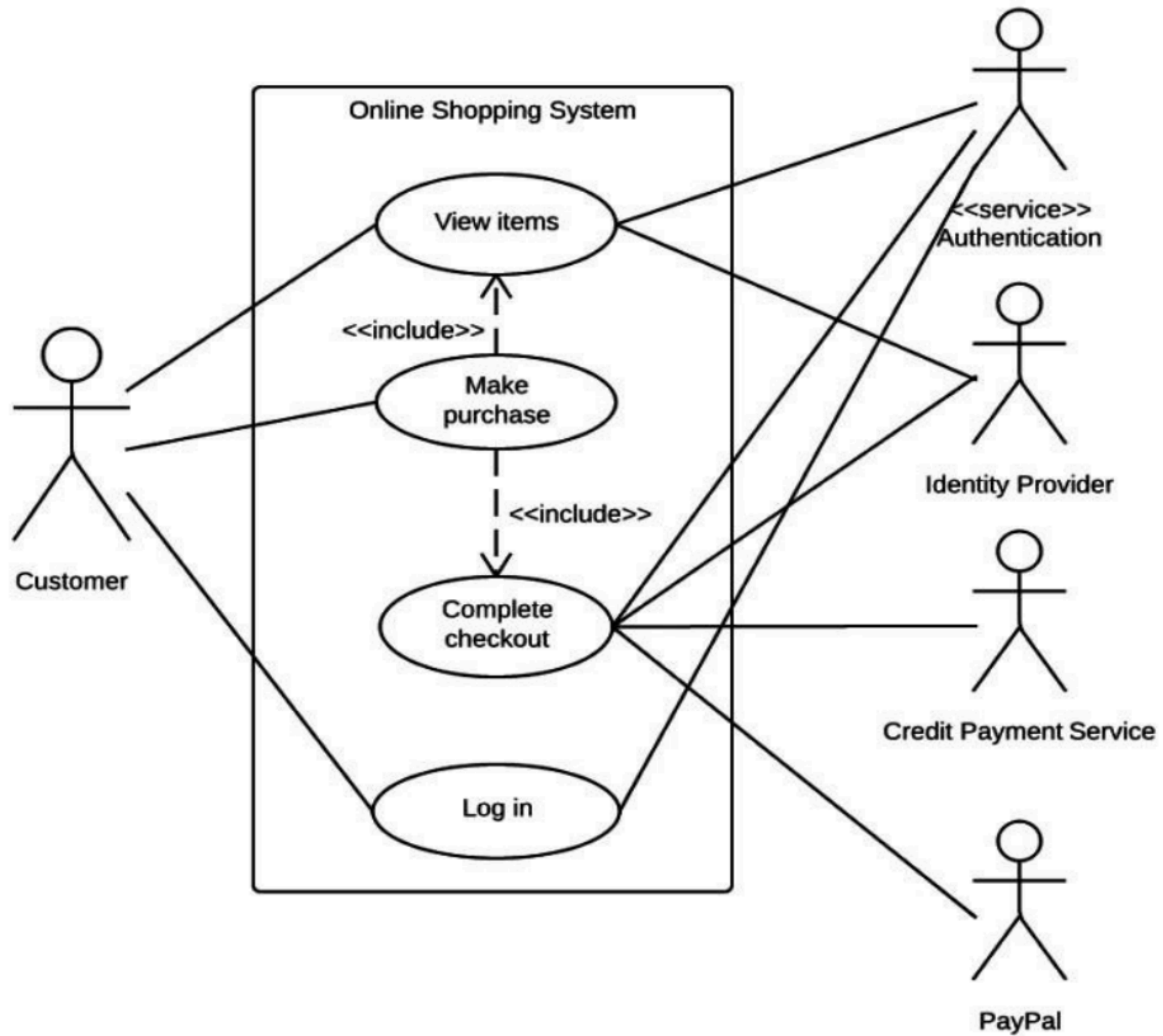
- Customer
- Admin

- **Use Cases:**

- Browse Products
- Add to Cart
- Checkout
- Manage Inventory (Admin)

- **Relations:**

- The Customer can browse products, add to the cart, and complete the checkout.
- The Admin can manage the inventory.



3.5.2 Class Diagrams:

In software engineering, a UML class diagram is a static structure diagram that graphically represents a system's classes, their attributes, operations (methods), and the relationships between them.

What a Class Diagram Shows?

Classes: Rectangles representing the fundamental building blocks of an object-oriented system, showing the system's structure.

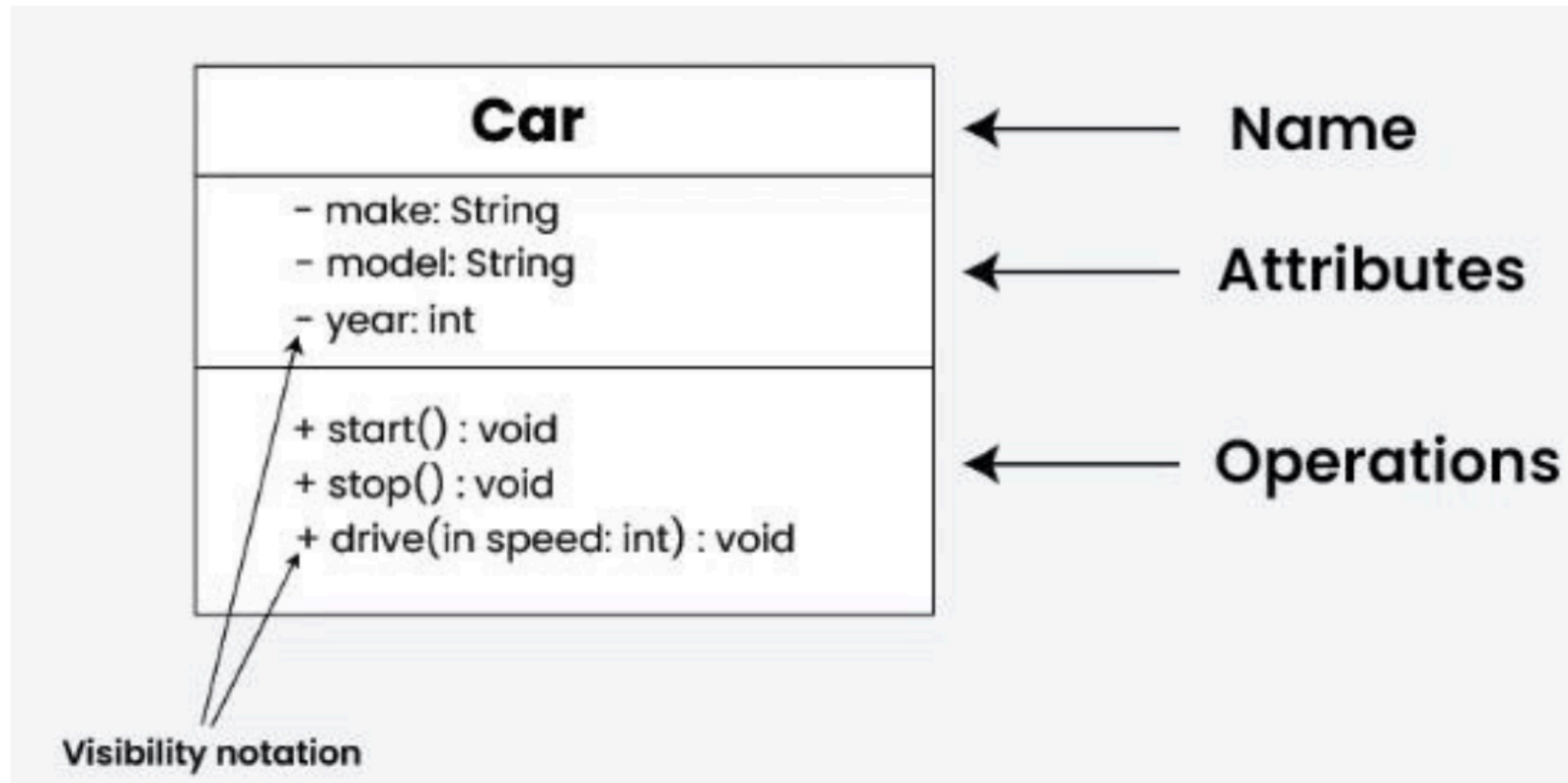
Attributes: The data or variables that a class holds, listed within the class box.

Operations (Methods): The functions or actions a class can perform, also listed within the class box.

Relationships: Different types of arrows and lines connecting classes to show how they interact, such as:

- **Association:** A general relationship between two classes.
- **Inheritance:** Shows a subclass inheriting properties and behaviors from a superclass.
- **Composition and Aggregation:** Represent "part-of" relationships, where one class is a part of another.

3.5.2 Class Diagrams:



3.5.5 Sequence Diagrams:

In software engineering, a sequence diagram is a type of UML interaction diagram that visualizes the sequence of messages exchanged between objects or components of a system to accomplish a specific task or scenario. It shows how different parts of a system interact with each other over time, illustrating the chronological order of these interactions.

Key Components of a Sequence Diagram

Lifelines: Vertical dashed lines representing the existence of an object or participant over time.

Messages: Horizontal arrows between lifelines, indicating the invocation of operations or the passing of data between objects.

Synchronous Message: A solid line with a filled arrowhead, representing a call where the sender waits for a reply.

Asynchronous Message: A solid line with an open arrowhead, where the sender does not wait for a response.

Return Message: A dashed line with an open arrowhead, indicating the completion of a synchronous message.

Activation Boxes: Thin, vertical rectangles on a lifeline, showing the period during which an object is actively executing an operation.

3.6 Testing :

- Software engineering testing is a crucial Software Development Lifecycle (SDLC) process that involves verifying and validating a software application to ensure it meets requirements, functions correctly, and is free of defects (bugs).
- Testing helps identify issues like functional errors, performance problems, and security vulnerabilities, ultimately ensuring the software is reliable, high-performing, and provides a positive user experience.
- It involves comparing actual software behavior to expected behavior and is done through manual or automated methods.

Meaning and purpose:

- Software testing is the process of evaluating software to find bugs, verify requirements are met, and ensure the product is high-quality, secure, efficient, and user-friendly before release.
- Its purpose is to reduce development costs by identifying issues early, enhance user satisfaction, increase confidence in the software's reliability and performance, and mitigate risks associated with defects.

3.6.2 Testing Based on Scope

Unit Testing: Developers test individual components or units of code to ensure they function correctly.

Integration Testing: Tests how different units or components work together after being integrated.

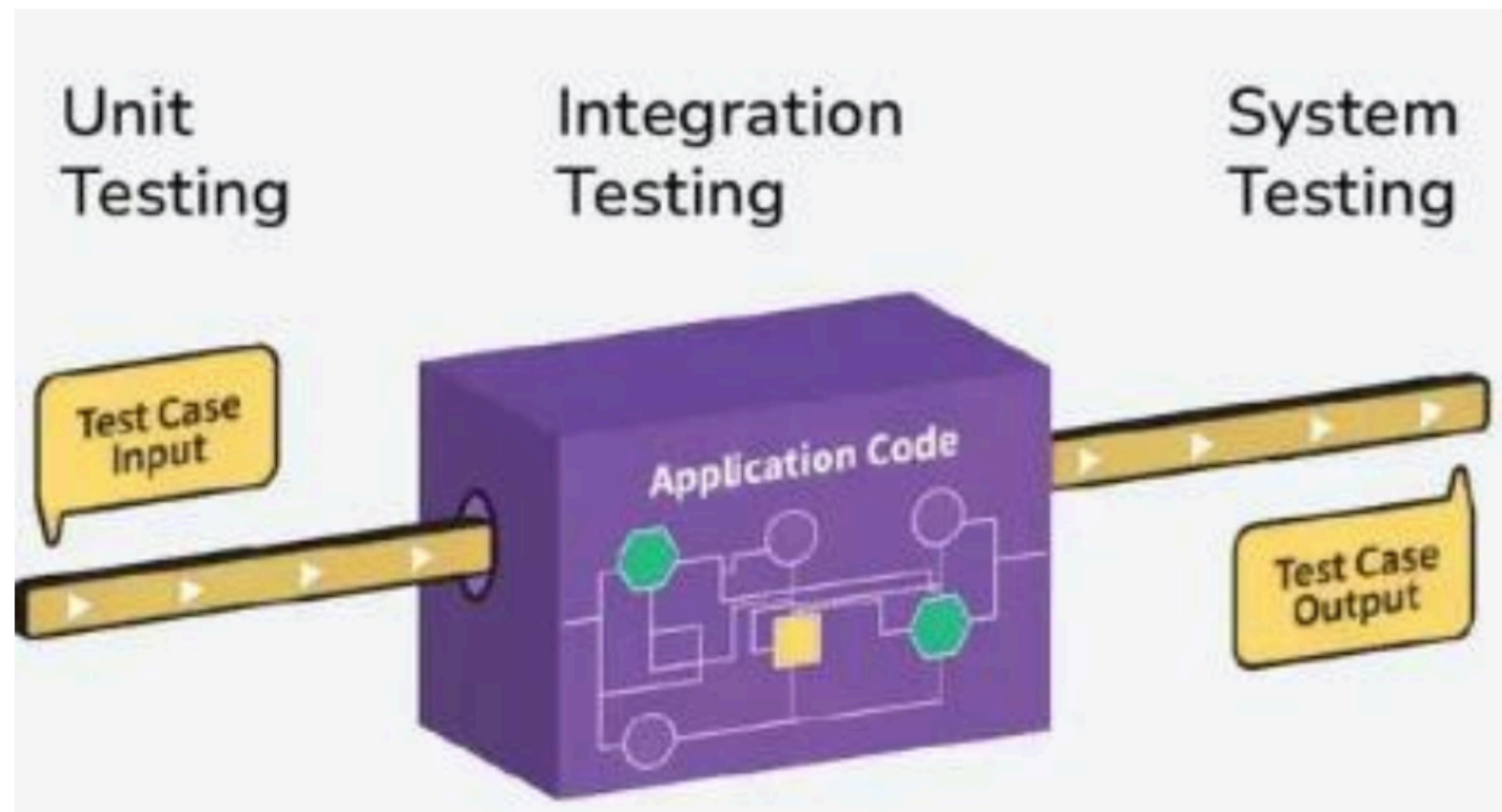
System Testing: Evaluates the complete, integrated software system to ensure it meets all specified requirements.

Acceptance Testing: Done by end-users or clients to verify that the software meets their business requirements and is ready for release.

3.6.3 Testing Based on Method

White Box Testing:

- White-box Testing is a Software Testing Technique that involves testing the internal structure and workings of a Software Application. The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.
- Requires knowledge of the internal code and design to design test cases that verify code logic and internal structure.



3.6.3 Testing Based on Method

Black Box Testing: Tests the software's functionality without knowledge of its internal code or structure, focusing on inputs and outputs.



➤ **Black-box and White-box:**

Parameters	Black Box Testing	White Box Testing
Definition	Black Box Testing is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	White Box Testing is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
Testing objectives	Black box testing is mainly focused on testing the functionality of the software, ensuring that it meets the requirements and specifications.	White box testing is mainly focused on ensuring that the internal code of the software is correct and efficient.
	Black box testing is generally used for testing the software at the functional level.	White box testing is used for testing the software at the unit level, integration level and system level.
Implementation	Implementation of code is not needed for black box testing.	Code implementation is necessary for white box testing.
Done By	Black Box Testing is mostly done by software testers.	White Box Testing is mostly done by software developers.

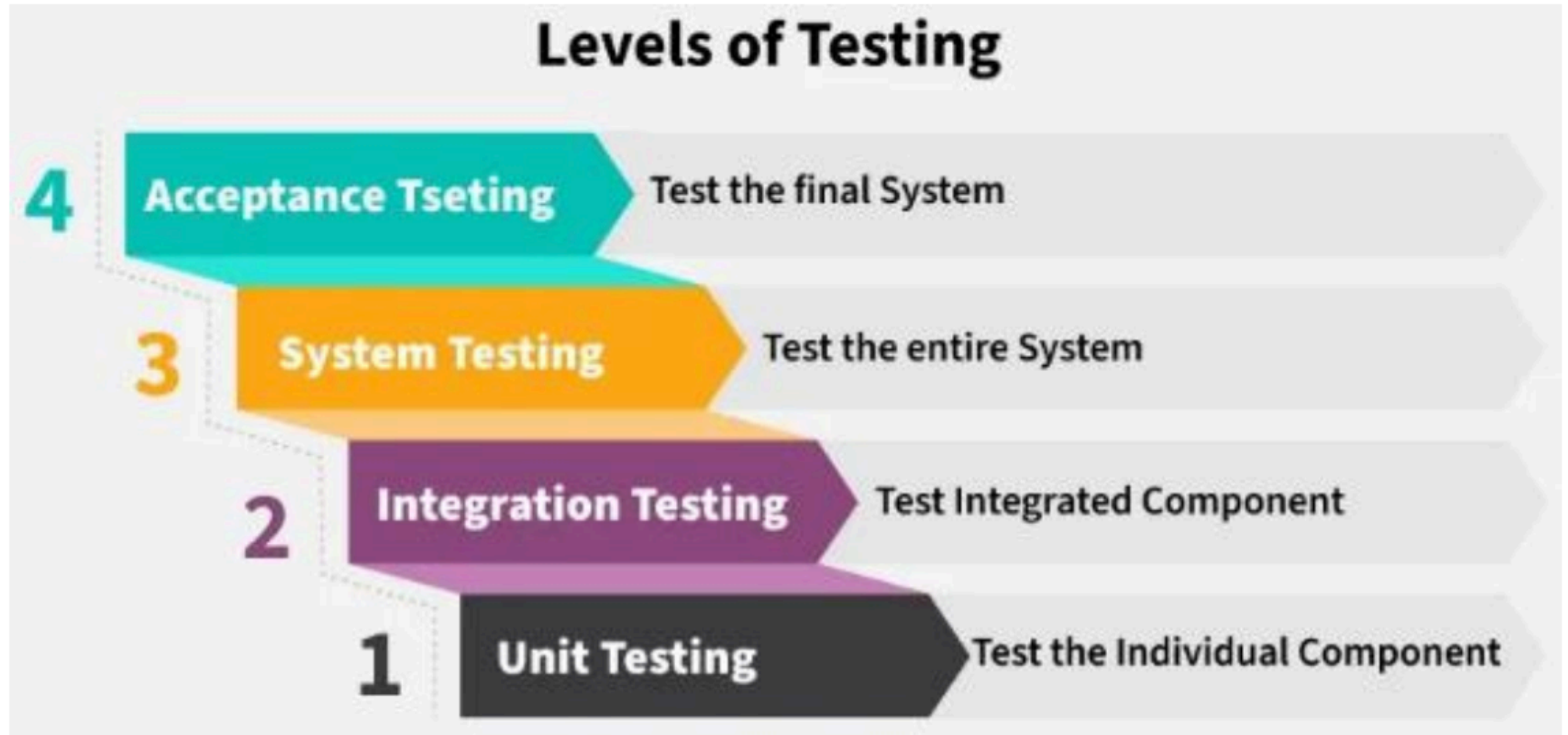
Terminology	Black Box Testing can be referred to as outer or external software testing.	White Box Testing is the inner or the internal software testing.
Testing Level	Black Box Testing is a functional test of the software.	White Box Testing is a structural test of the software.
Testing Initiation	Black Box testing can be initiated based on the requirement specifications document.	White Box testing of software is started after a detail design document.
Programming	No knowledge of programming is required.	It is mandatory to have knowledge of programming.
Testing Focus	Black Box Testing is the behavior testing of the software.	White Box Testing is the logic testing of the software.

➤ **Static and Dynamic testing:**

Parameters	Static Testing	Dynamic Testing
Definition	Static testing is performed to check the defects in the software without actually executing the code.	Dynamic testing is performed to analyze the dynamic behavior of the code.
Objective	The objective is to prevent defects.	The objective is to find and fix defects.
Stage of execution	It is performed at the early stage of software development.	It is performed at the later stage of the software development.
Code Execution	In static testing, the whole code is not executed.	In dynamic testing, the whole code is executed.
Before/ After Code Deployment	Static testing is performed before code deployment.	Dynamic testing is performed after code deployment.
Cost	Static testing is less costly.	Dynamic testing is highly costly.

Parameters	Static Testing	Dynamic Testing
Documents Required	Static Testing involves a checklist for the testing process.	Dynamic Testing involves test cases for the testing process.
Time Required	It generally takes a shorter time.	It usually takes a longer time as it involves running several test cases.
Bugs	It can discover a variety of bugs.	It exposes the bugs that are explorable through execution hence discovering only a limited type of bugs.
Statement Coverage	Static testing may complete 100% statement coverage incomparably in less time.	Dynamic testing only achieves less than 50% coverage.
Techniques	It includes Informal reviews, walkthroughs, technical reviews, code reviews, and inspections.	It involves functional and non-functional testing.
Example	It is a verification process.	It is a validation process.

➤ **Level of testing:**



1. Unit Testing

- Unit Testing is the first step in testing your software. It focuses on checking individual components or functions of the application to make sure they work correctly on their own. The goal here is to catch any issues early before those small components are integrated with the rest of the system.
- Unit tests help developers spot bugs early in the development process, making it easier and quicker to fix them. It's the first layer of defense to verify that each part of the application performs as expected.

Mainly features are:

- Testing individual functions or components.
- Ensuring correct behavior at the smallest level.
- Catching errors early.

2. Integration Testing

- After unit testing, Integration Testing takes over. This level checks how different modules or components of the software work together. It's important because even if individual parts work perfectly, they might face issues when interacting with one another.
- Integration testing ensures that data flows correctly between modules and that they communicate seamlessly. It helps catch problems that might arise when different parts of the system interact.

3. System testing

System Testing is when you test the software as a system. This stage checks whether the entire system functions as expected in a real-world environment. It includes both functional and non-functional tests to ensure that the software meets customer needs.

- Full end-to-end testing of the software.
- Verifying both functional and non-functional requirements.
- Testing the software's behavior in real-world conditions.

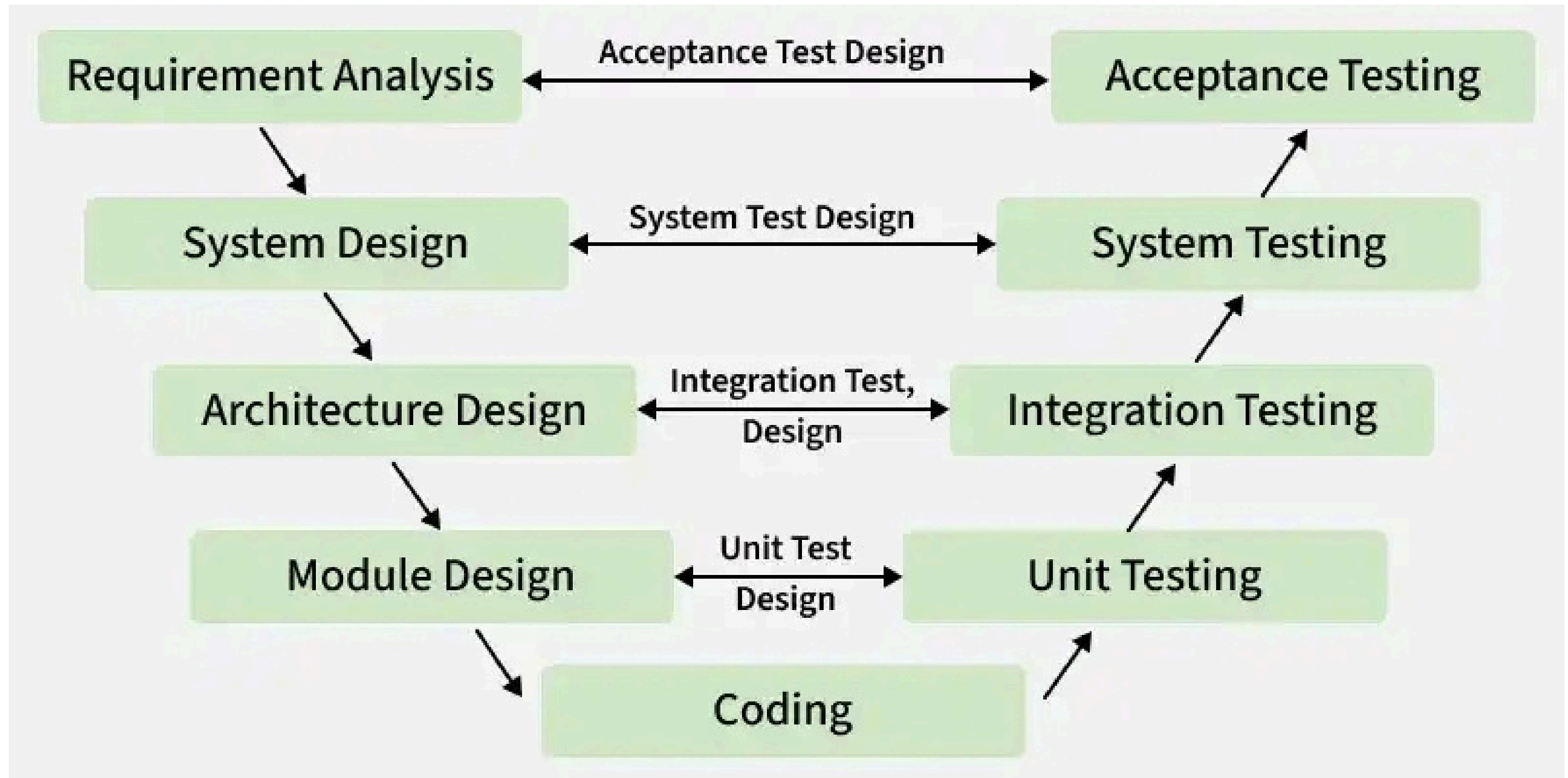
4. Acceptance Testing

- Acceptance Testing, also known as User Acceptance Testing (UAT), is the final test before releasing the software to the end-users. In this phase, the customer or end-users verify if the software meets their needs and expectations.
- UAT is crucial because it verify the software is ready for production. It's the last chance to catch any overlooked issues before deployment. If the software passes this stage, the customer gives the green light for release.
 - Validating the software against business requirements.
 - Ensuring the software meets customer expectations.
 - Getting final approval from the customer.

V-model:

- **The V-Model is a software development lifecycle (SDLC) model that extends the waterfall model by creating a V-shaped process with closely associated verification and validation phases.**
- **On the left side of the "V," development stages like requirements analysis and design are performed, while on the right side, corresponding testing phases (such as unit testing, integration testing, and acceptance testing) are planned and executed in parallel, ensuring quality throughout the process.**

3.6.4 V-Model



1. V-Model Verification Phases

- This is where the process begins. The first step is to gather and understand the customer's needs for the software. The goal is to define the scope of the project clearly to make sure everyone is on the same page.
- It involves a static analysis technique (review) done without executing code. It is the process of evaluation of the product development phase to find whether specified requirements are met.

There are several Verification phases in the V-Model:

1. Business Requirement Analysis

- This is the first step of the designation of the development cycle where product requirement needs to be cured from the customer's perspective. These phases include proper communication with the customer to understand their requirements.
- These are the very important activities that need to be handled properly, as most of the time customers do not know exactly what they want, and they are not sure about it at that time then we use an acceptance test design planning which is done at the time of business requirement it will be used as an input for acceptance testing.

2. System Design

- In this phase, the overall structure of the software is planned out. The team develops both the high-level design (how the system will be structured) and detailed design (how the individual components will work).
- Design of the system will start when the overall we are clear with the product requirements, and then need to design the system completely. This understanding will be at the beginning of complete under the product development process. these will be beneficial for the future execution of test cases.

3. Architectural Design

- In this stage, architectural specifications are comprehended and designed. Usually, several technical approaches are put out, and the ultimate choice is made after considering both the technical and financial viability. The system architecture is further divided into modules that each handle a distinct function. Another name for this is High-Level Design (HLD).
- At this point, the exchange of data and communication between the internal modules and external systems are well understood and defined. During this phase, integration tests can be created and documented using the information provided.

4. Module Design: This phase, known as Low-Level Design (LLD), specifies the comprehensive internal design for every system module. Compatibility between the design and other external systems as well as other modules in the system architecture is crucial. Unit tests are a crucial component of any development process since they assist in identifying and eradicating the majority of mistakes and flaws at an early stage. Based on the internal module designs, these unit tests may now be created.

5. Coding Phase

- This is where the software is actually built. Developers write the code based on the design created in the previous phase.
- The Coding step involves writing the code for the system modules that were created during the Design phase.
- The system and architectural requirements are used to determine which programming language is most appropriate.
- The coding standards and principles are followed when performing the coding. Before the final build is checked into the repository, the code undergoes many code reviews and is optimized for optimal performance.

2. V-Model Validation Phases

It involves dynamic analysis techniques (functional, and non-functional), and testing done by executing code. Validation is the process of evaluating the software after the completion of the development phase to determine whether the software meets the customer's expectations and requirements.

1. Unit Testing: In Unit testing, unit Test Plans are developed during the module design phase. These Unit Test Plans are executed to eliminate bugs in code or unit level.

2. Integration testing: After completion of unit testing Integration testing is performed. In integration testing, the modules are integrated and the system is tested. Integration testing is performed in the Architecture design phase. This test verifies the communication of modules among themselves.

3. System Testing:

System testing tests the complete application with its functionality, inter-dependency, and communication. It tests the functional and non-functional requirements of the developed application.

4. User Acceptance Testing (UAT): User Acceptance Testing (UAT) is performed in a user environment that resembles the production environment. UAT verifies that the delivered system meets the user's requirement and the system is ready for use in the real world.